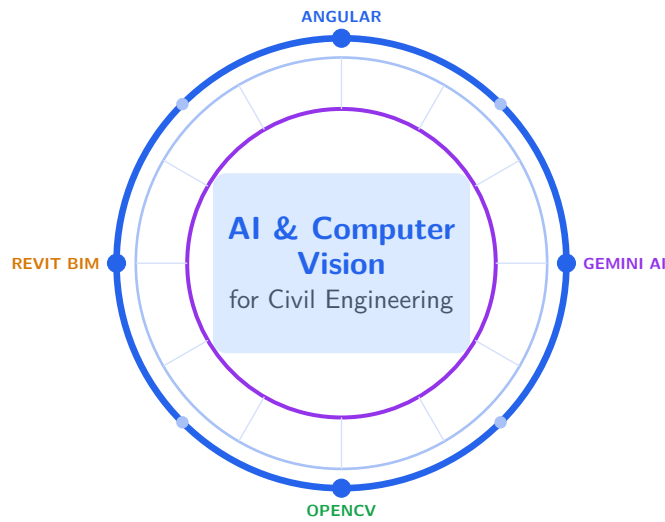


Rebar Analysis

The Complete Engineering Codebook



A comprehensive guide to the architecture, algorithms, and code behind the automated rebar inspection and quality control platform.

Frontend

Angular

AI Backend

Python + Gemini

BIM Integration

Autodesk Revit

Architected & Engineered by **Chaitanya Patil**

A Note Before We Begin

This codebook is written for **anyone** who wishes to understand the RebarAnalysis project deeply. No prior programming experience is assumed. Every concept, every line of code, and every design decision is explained from first principles using clear language, analogies, and visual diagrams. By the end of this book, you will not only understand what the software does, but exactly *how* and *why* it does it.

"Clarity is the soul of great engineering."

LIVE PLATFORM DEPLOYMENT

rebaranalysis.vercel.app

OPEN SOURCE REPOSITORY

[XeCipher/RebarAnalysis](https://github.com/XeCipher/RebarAnalysis)

How to Read This Book

- ★ **Key Concept** boxes: Core ideas explained simply
- 📖 **Analogy** boxes: Real-world comparisons
- 📌 **Important Note** boxes: Things to watch out for
- ✓ **Technical Insight** boxes: In-depth engineering detail
- ☆ **Deep Dive** boxes: Advanced explanation
- 💡 **Did You Know?** boxes: Interesting facts
- 📄 Code files are shown in [blue bordered boxes](#)
- ❖ Diagrams appear throughout each chapter

Contents

1	The Vision and Overview	5
1.1	The Problem: A Day on a Construction Site	5
1.2	The Solution: RebarAnalysis	6
1.3	The Four Superpowers	6
1.4	The Three Brains	7
1.5	The Technology Stack at a Glance	8
2	System Architecture	9
2.1	The Complete Data Flow	9
2.2	The Frontend-Backend Split: Why Two Separate Systems?	10
2.3	The API Contract: How Frontend and Backend Communicate	11
2.4	The Calibration Problem: Pixels vs Millimeters	12
2.5	Deployment Architecture: Local vs Cloud	12
3	The Frontend Experience (Angular)	13
3.1	What is Angular and Why Use It?	13
3.2	The Entry Point: <code>index.html</code>	14
3.2.1	How Browsers Load the Page	14
3.3	The Bootstrapper: <code>main.ts</code>	15
3.4	The Application Config: <code>app.config.ts</code>	15
3.5	The Heart: <code>app.component.ts</code>	16
3.5.1	Understanding State Management	16
3.5.2	The Step-by-Step Workflow	17
3.5.3	The Image Click Handler: Scaling Screen Coordinates to Real Image Coordinates	18
3.5.4	Image Upload Handling	19
3.5.5	The Analysis Trigger: Submitting to the Backend	20
3.6	The AI Whisperer: <code>gemini.service.ts</code>	21
3.6.1	What is a Large Language Model?	22
3.6.2	Prompt Engineering: Talking to AI Precisely	22
3.6.3	The Side View Prompt	23
3.6.4	Calling the Gemini API	23
3.7	The Grader: <code>scoring.service.ts</code>	25
3.7.1	The Scoring Weights	25
3.7.2	The Tolerance Classification	25
3.8	The Alert System: Email Notifications	27
3.9	The Report Generator	28
4	The Backend Brain (Python & OpenCV)	29
4.1	What is OpenCV?	29
4.2	The Server Entry: <code>app.py</code>	30

4.2.1	CORS: The Security Policy That Must Be Bypassed	32
4.2.2	The Email System with Threading	32
4.3	Computer Vision Core: <code>analysis_service.py</code>	33
4.3.1	Step 1: Extracting the Region of Interest (ROI)	33
4.3.2	Step 2: Converting to Grayscale	34
4.3.3	Step 3: Canny Edge Detection	35
4.3.4	Step 4: Ray Casting – The Core Algorithm	36
4.3.5	Calculating Spacing Between Rods	38
4.4	Side Elevation Analysis: <code>side_view_service.py</code>	38
4.4.1	The Challenge: Variable Lighting	39
4.4.2	CLAHE: Adaptive Contrast Enhancement	39
4.4.3	Morphological Operations: Finding Horizontal Lines	39
4.4.4	Line Fitting with Polynomial Regression	41
4.5	The Email Service: <code>email_service.py</code>	42
5	BIM Integration (Autodesk Revit)	45
5.1	What is Building Information Modelling?	45
5.2	Dynamo: The Bridge Between Python and Revit	45
5.3	The Highlight JSON File	46
5.4	The Revit Highlight Script: <code>rodhighlight.py</code>	46
5.5	The Data Export Script: <code>write_report.py</code>	49
6	Execution and Deployment	51
6.1	The Launcher: <code>run.bat</code>	51
6.1.1	Understanding the Batch Script	53
6.2	The Backend Dependencies: <code>requirements.txt</code>	53
6.2.1	Why Gunicorn in Production?	53
6.3	Cloud Deployment Architecture	53
6.3.1	Vercel: Hosting the Angular Frontend	54
6.3.2	Render: Hosting the Python Backend	54
6.4	Environment Configuration: <code>environment.ts</code>	55
6.5	The Complete File Structure	55
7	Data Models and TypeScript Interfaces	57
7.1	What is TypeScript?	57
7.2	The Core Data Models	57
8	Algorithms Deep Dive	60
8.1	Euclidean Distance	60
8.2	Median vs Mean: Why We Use Median	61
8.3	Otsu's Thresholding: Automatic Edge Detection	61
8.4	The Bilateral Filter: Mathematics of Smart Blurring	62
9	Security, Best Practices, and Code Quality	63
9.1	Environment Variables: Never Hardcode Secrets	63
9.2	Input Validation	64
9.3	Error Handling Strategy	64
9.4	TypeScript Strict Mode	65
9.5	Code Organization Principles	66
10	Testing, Validation, and Quality Assurance	67

10.1 The Test Dataset	67
10.2 Backend Unit Tests	68
10.3 Calibration Accuracy Validation	69
11 Complete User Workflow	70
11.1 Step-by-Step Walkthrough	70
11.1.1 Step 1: Landing on the Application	70
11.1.2 Step 2: Uploading the Site Photo	70
11.1.3 Step 3: Uploading the Blueprint	70
11.1.4 Step 4: Selecting the View Type	70
11.1.5 Step 5: Placing Rod Markers	71
11.1.6 Step 6: Setting the Calibration Reference	71
11.1.7 Step 7: AI Blueprint Analysis	71
11.1.8 Step 8: Python Backend Analysis	71
11.1.9 Step 9: Score Calculation and Report	72

CHAPTER 1

The Vision and Overview

Every great piece of engineering starts with a problem worth solving. This chapter answers the most fundamental question: **What exactly is RebarAnalysis, why does it exist, and what does it set out to accomplish?** We will walk through the real-world problem it solves, the solution it delivers, and the big picture of how its three major parts work together.

1.1 The Problem: A Day on a Construction Site

Imagine standing on a real construction site. A massive reinforced concrete column is about to be poured. Before the liquid concrete is pumped in and solidifies forever, an engineer must verify that the internal steel skeleton, called the **rebar cage**, matches the architect's design blueprint exactly.

This is not a trivial task. The engineer must:

- Walk around the cage with a measuring tape
- Count every single steel bar
- Measure the gap between each bar
- Compare each measurement against a paper drawing
- Write all findings into a manual inspection report
- Photograph the site and attach photos to the document
- Get a supervisor to sign off

On a complex column with 20 or 30 bars, this takes hours. And it happens on *every single column* of a building that might have thousands of columns. Errors slip through. Measurements are estimated. Documentation is inconsistent.

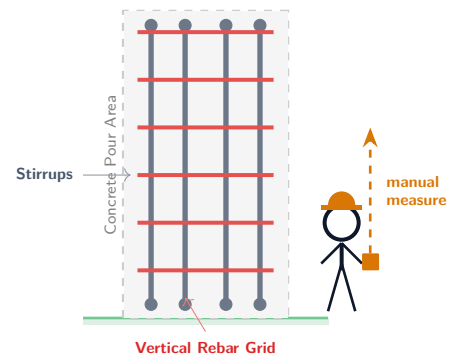


Figure 1.1: *
Manual inspection of a rebar cage

Critical Point

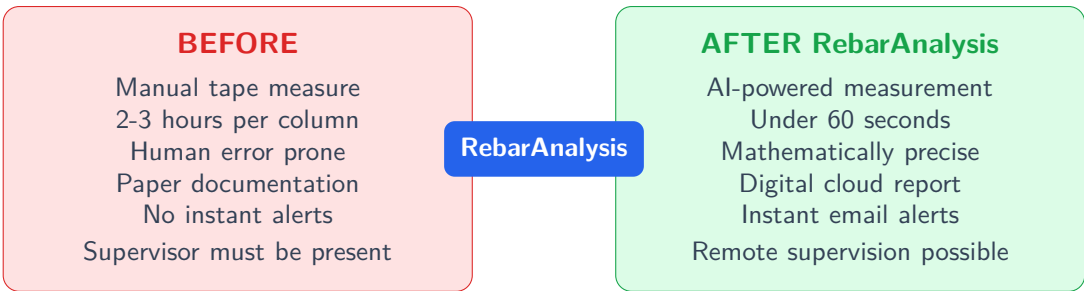
In civil engineering, a missing rebar or an incorrectly spaced bar can lead to structural failure. Buildings can crack, floors can collapse, and lives can be lost. This is not a domain where “close enough” is ever acceptable.

1.2 The Solution: RebarAnalysis

RebarAnalysis replaces this error-prone, time-consuming manual process with an intelligent, automated platform. The engineer simply:

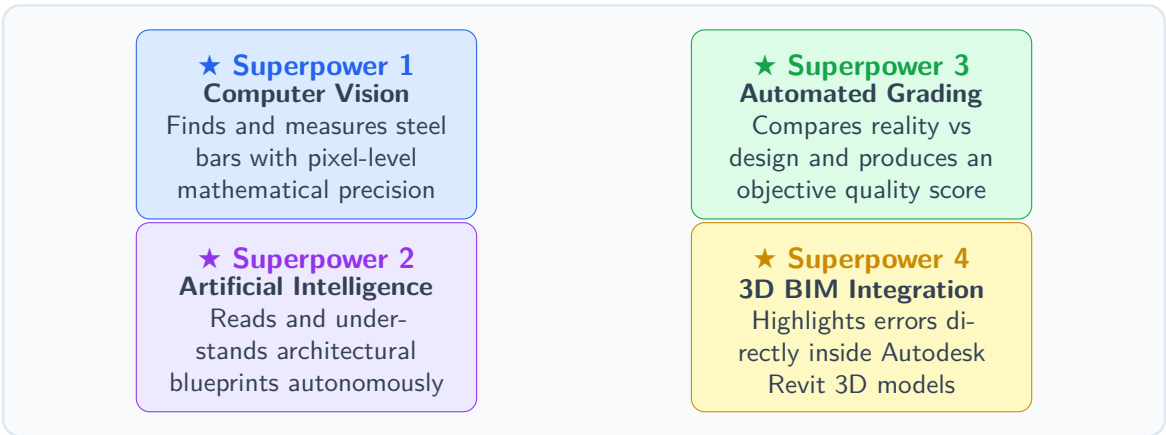
1. Takes a **photograph** of the rebar cage on site
2. Takes a **photo** or scan of the architectural drawing
3. Uploads both to the RebarAnalysis web application

The system then does everything else: it finds the steel bars in the photograph, measures their exact positions in real-world millimeters, reads the design drawing using AI, compares the two, generates a quality score, and if the work is dangerous, instantly fires off an alert email to the project manager.



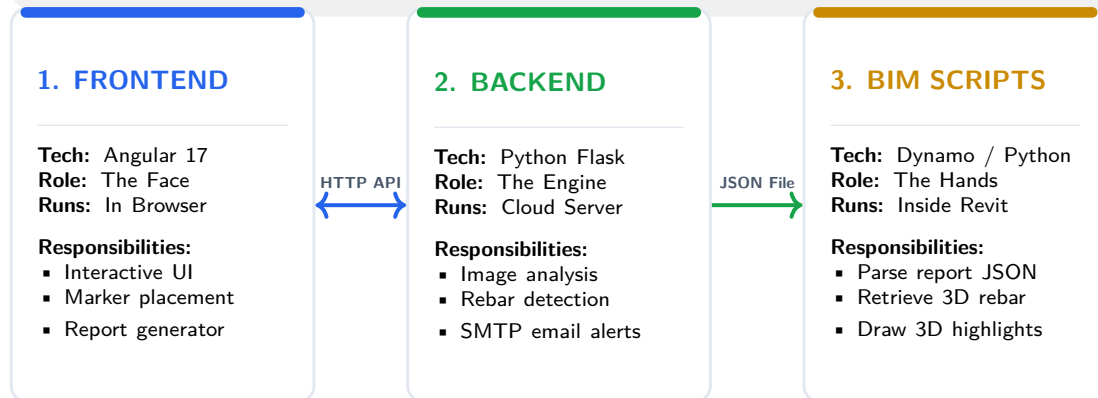
1.3 The Four Superpowers

RebarAnalysis is built around four core technological capabilities:



1.4 The Three Brains

The entire system is divided into three specialized modules, each doing one job exceptionally well. Think of it like the human body: the *face* that interacts with the world, the *brain* that crunches data, and the *hands* that act in the 3D environment.



★ Car Analogy

Think of the **Frontend** as the dashboard and steering wheel of a car: it is what the driver (engineer) interacts with. The **Backend** is the engine and transmission: the driver never sees it, but it does all the heavy work. The **BIM Scripts** are like the GPS display that shows your destination in 3D space.

1.5 The Technology Stack at a Glance

Layer	Technology	What It Does
Frontend Framework	Angular 17	Builds the web UI with reactive data binding
Frontend Styling	Tailwind CSS	Provides pre-built design utilities
AI Language Model	Google Gemini	Reads and interprets blueprint drawings
Backend Language	Python 3.11	Runs mathematical image analysis
Backend Framework	Flask	Serves the Python logic via HTTP
Computer Vision	OpenCV	Detects shapes, edges, and measurements in photos
Math Library	NumPy	Handles matrix operations on image data
3D BIM	Autodesk Revit	Industry-standard 3D building model
BIM Scripting	Dynamo + Python	Automates actions inside Revit
Frontend Hosting	Vercel	Global CDN hosting for Angular app
Backend Hosting	Render	Cloud server for Python processing

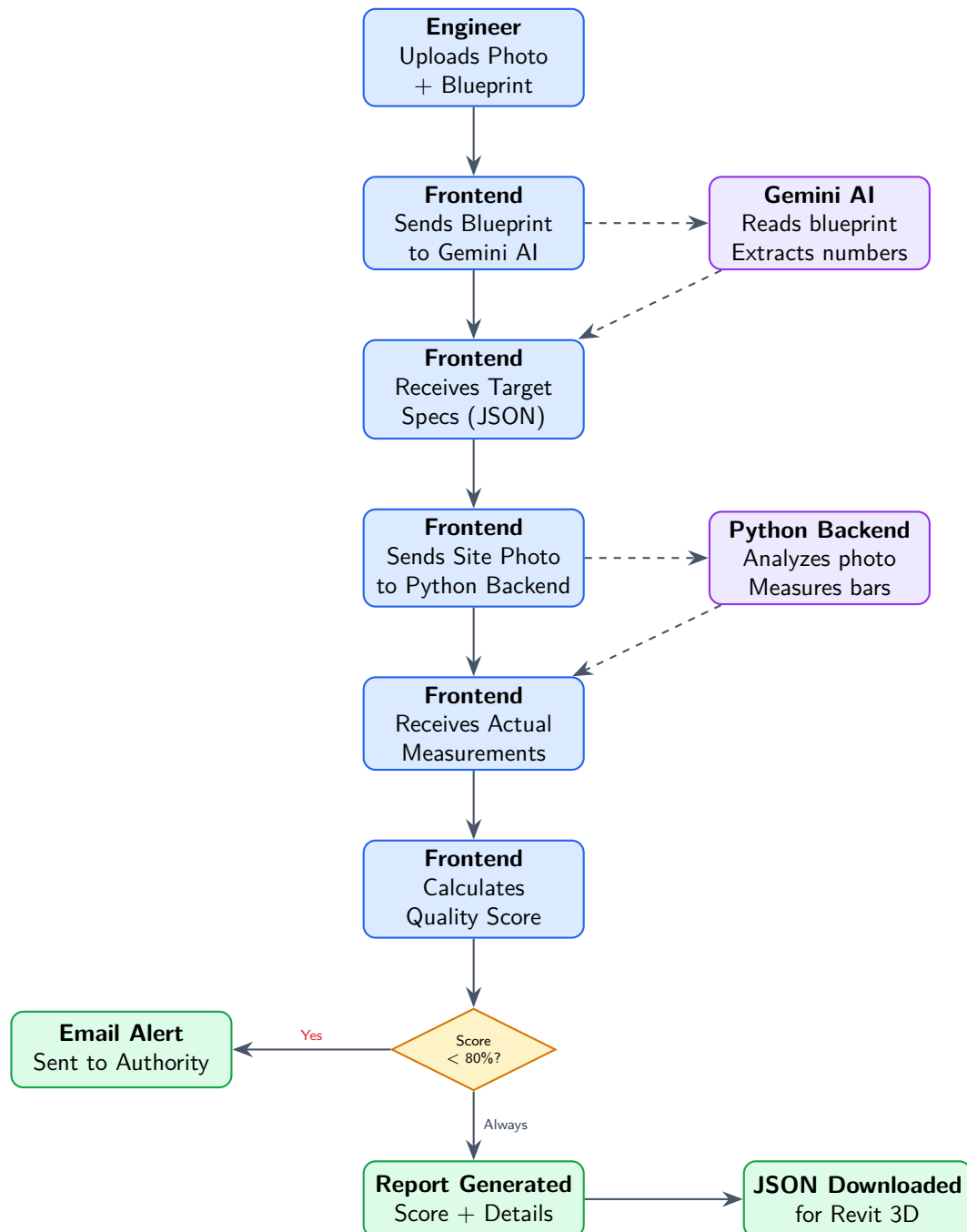
CHAPTER 2

System Architecture

Before touching a single line of code, a great engineer always *draws the map*. In this chapter, we will look at the complete architecture of the RebarAnalysis system: how data flows from the engineer's phone through multiple systems, what decisions are made at each step, and why we designed things the way we did.

2.1 The Complete Data Flow

When a user uploads a photo and a blueprint, a carefully choreographed sequence of events begins. Let us trace the complete journey of a single quality check from start to finish.



2.2 The Frontend-Backend Split: Why Two Separate Systems?

Important Note

This is one of the most important architectural decisions in the entire project. Understanding *why* we split the system matters more than understanding *how*.

When processing a photograph, we need to perform millions of mathematical operations. An

image that is 4000 × 3000 pixels has **12 million individual data points**. For each pixel, we might need to perform 10 or 20 mathematical operations to detect edges, calculate gradients, and find circles.

That is potentially **240 million operations** for a single analysis.

If done in the browser...

The user's phone/laptop CPU would need to run 240 million operations in JavaScript. JavaScript is interpreted, not compiled. A modern phone might take 30-120 seconds. The browser tab would freeze and become unresponsive. The user experience would be terrible.



With Python on a server...

The server runs compiled C++ code through Python's OpenCV library. Our server's dedicated CPU handles 240 million operations in under 2 seconds. The user's phone just shows a loading spinner while the heavy lifting is done elsewhere.

2.3 The API Contract: How Frontend and Backend Communicate

The Frontend and Backend communicate via a standard web protocol called **HTTP REST API**. The Frontend sends a **POST** request containing the image data, and the Backend responds with a **JSON** object containing the measurement results.

□ API Contract: Frontend Request to Backend

```

1  # What the Frontend sends (HTTP POST to /analyze):
2  {
3    "image": "<base64 encoded image data>",
4    "rod_points": [[450, 380], [720, 380], [990, 380]],
5    "view_type": "top" # or "side"
6  }
7
8  # What the Backend returns (HTTP 200 JSON response):
9  {
10   "rods": [
11     {"center": [450, 380], "radius_px": 28.5, "radius_mm": 8.1},
12     {"center": [720, 380], "radius_px": 27.9, "radius_mm": 7.9},
13     {"center": [990, 380], "radius_px": 28.2, "radius_mm": 8.0}
14   ],
15   "spacings_mm": [75.3, 74.8, 75.1],
16   "calibration_px_per_mm": 3.52
17 }
```

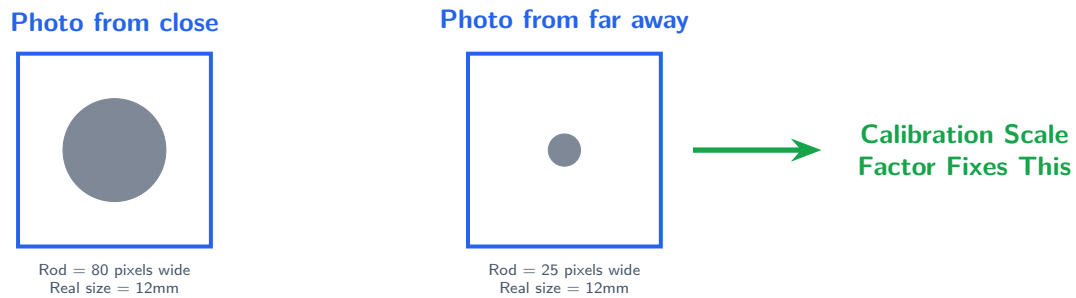
★ What is Base64?

When you send a file over the internet in a text-based format (like JSON), you cannot include raw binary image data. **Base64 encoding** is a way of converting binary data (the raw bytes of an image) into a long string of regular text characters. Think of it as translating a photograph into a very long paragraph of text that computers can transmit, then translate back into the photograph on the other side.

2.4 The Calibration Problem: Pixels vs Millimeters

One of the most important architectural concepts in the project is **calibration**: converting from pixels (what the computer sees) to millimeters (what the engineer needs).

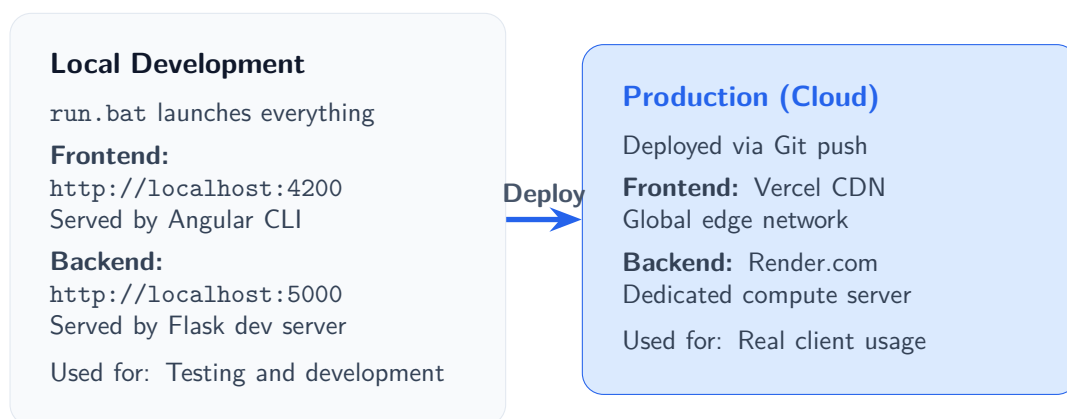
A pixel has no inherent size. The same steel bar that is 12mm wide might appear as 30 pixels wide on one photo and 100 pixels wide on another photo taken from a different distance.



The solution is elegant: the user places a marker on a known reference point (like the outer edge of the column, which has a known size from the blueprint). This gives the system a **px_per_mm** ratio (pixels per millimeter). Every subsequent measurement is divided by this ratio to get real-world millimeters.

$$\text{Real Distance (mm)} = \frac{\text{Pixel Distance (px)}}{\text{Scale Factor (px/mm)}}$$

2.5 Deployment Architecture: Local vs Cloud



CHAPTER 3

The Frontend Experience (Angular)

The Frontend is everything the user sees and touches. It is built with **Angular**, a powerful framework created by Google for building large-scale web applications. In this chapter, we will go through every significant file, understand what it does, how it works, and why it was built that way.

3.1 What is Angular and Why Use It?

Components

Self-contained building blocks. Each component has its own HTML template, TypeScript logic, and CSS styling.

Services

Shared logic modules that any component can use.
Example: the Gemini API call is a service used by multiple components.

Data Binding

When a variable changes in TypeScript, Angular automatically updates the HTML in the browser. No manual DOM manipulation.

★ Why Angular over plain HTML?

Plain HTML and JavaScript become extremely hard to manage once an application grows. You end up with thousands of lines of code trying to manually update the screen when data changes. Angular's **component system** and **automatic data binding** mean that when a variable changes in the code, the screen updates by itself. It is the difference between manually updating a scoreboard by hand versus having it connected to an automatic computer system.

3.2 The Entry Point: `index.html`

Every website has to start somewhere. When a user types the URL into their browser and hits Enter, the server delivers one file: `index.html`.

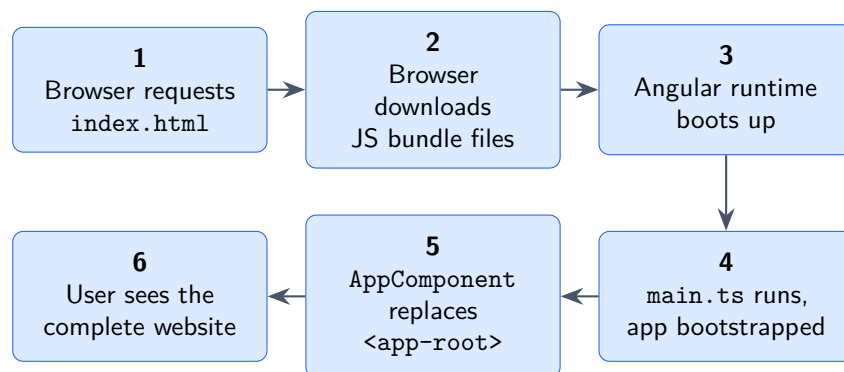
□ frontend/src/index.html

```
1 <!doctype html>
2 <html lang="en">
3 <head>
4   <meta charset="utf-8">
5   <meta name="viewport" content="width=device-width, initial-
6     scale=1">
7   <title>RebarAnalysis - AI Quality Control</title>
8
9   <!-- Open Graph / Social Media Preview Tags -->
10  <meta property="og:title" content="RebarAnalysis" />
11  <meta property="og:description" content="AI-powered rebar
12    quality control" />
13
14  <!-- Favicon -->
15  <link rel="icon" type="image/x-icon" href="favicon.ico">
16
17  <!-- Tailwind CSS (via CDN or build) -->
18  <link rel="stylesheet" href="styles.css">
19 </head>
20 <body>
21   <!-- This single tag is where Angular mounts the entire
22     application -->
23   <app-root></app-root>
24
25   <!-- Angular compiles into bundle scripts loaded here -->
26 </body>
27 </html>
```

☆ The Magic Tag: `<app-root>`

The most important line in `index.html` is `<app-root></app-root>`. This is a **custom HTML tag** that Angular registers. When the browser sees this tag, Angular's runtime boots up, finds the component that is registered under the selector `app-root`, and injects that component's entire HTML template inside this tag. The **entire application** lives inside this one tag.

3.2.1 How Browsers Load the Page



3.3 The Bootstrapper: main.ts

`main.ts` is the very first TypeScript file that runs. Its job is minimal but critical: it tells Angular which component is the *root* of the application.

❏ frontend/src/main.ts

```
1 import { bootstrapApplication } from '@angular/platform-browser';
2 import { AppComponent } from '../app/app.component';
3 import { appConfig } from '../app/app.config';
4
5 // This single line starts the entire Angular application.
6 // It mounts AppComponent as the root of the application tree.
7 bootstrapApplication(AppComponent, appConfig)
8   .catch((err) => console.error(err));
```

❖ bootstrapApplication vs older module approach

In older versions of Angular (before v16), you would have a file called `app.module.ts` that listed every component and service. Angular 17 (used here) uses a **standalone component** approach: each component declares its own dependencies. `bootstrapApplication` replaces the old module-based boot.

3.4 The Application Config: app.config.ts

This file configures application-wide settings, such as routing (which component to show for which URL).

❏ frontend/src/app/app.config.ts

```
1 import { ApplicationConfig } from '@angular/core';
2 import { provideRouter } from '@angular/router';
3 import { provideHttpClient } from '@angular/common/http';
4 import { routes } from '../app.routes';
5
6 export const appConfig: ApplicationConfig = {
7   providers: [
8     // Enable routing (URL navigation between pages)
9     provideRouter(routes),
10
11     // Enable HTTP requests (for talking to backend and Gemini AI)
12     provideHttpClient(),
13   ]
14 };
```

- `provideRouter(routes)`: Enables Angular's navigation system. When the user types a different URL or clicks a link, Angular shows a different component without reloading the whole page.

- `provideHttpClient()`: Makes the `HttpClient` service available everywhere. This is the tool that sends HTTP requests to our Python backend and to the Gemini AI API.

3.5 The Heart: `app.component.ts`

This is the most important and complex file in the entire frontend. It manages the application's **state** (what is currently happening), handles all user interactions, coordinates calls to services, and orchestrates the entire analysis workflow.

3.5.1 Understanding State Management

The word *state* refers to all the data that describes what is currently happening in the application. Think of state like a whiteboard in a room: you write on it what is currently true, and when something changes, you erase and rewrite.

Application State Whiteboard

1. SESSION & WORKSPACE

currentStep:1

isLoading:false

mode:"rods"

uploadedImg:File

blueprtImg:File

2. WORKSPACE DATA

rodPoints:[x,y],...

calibPoints:[p1,p2]

pxPerMm:3.52

3. ENGINE & AI RESULTS

geminiSpecs:{count,rad,...}

backResults:[...]

finalScore:87.5

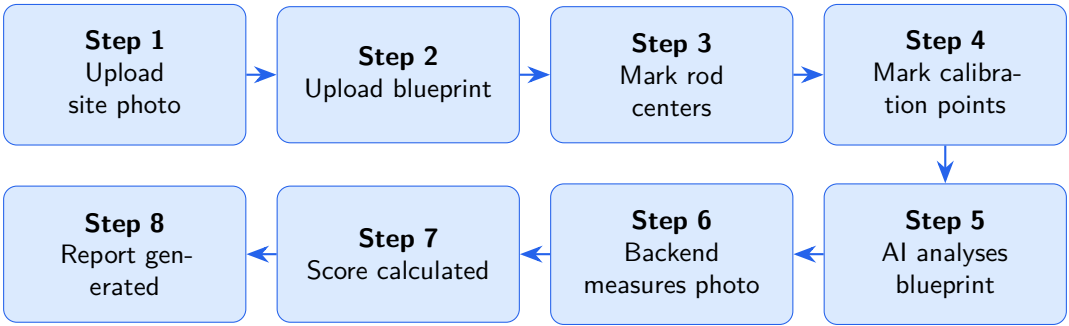
scoreStatus:"Good"

reportData:[...]

Angular automatically updates the UI and dependencies whenever the Application State changes.

3.5.2 The Step-by-Step Workflow

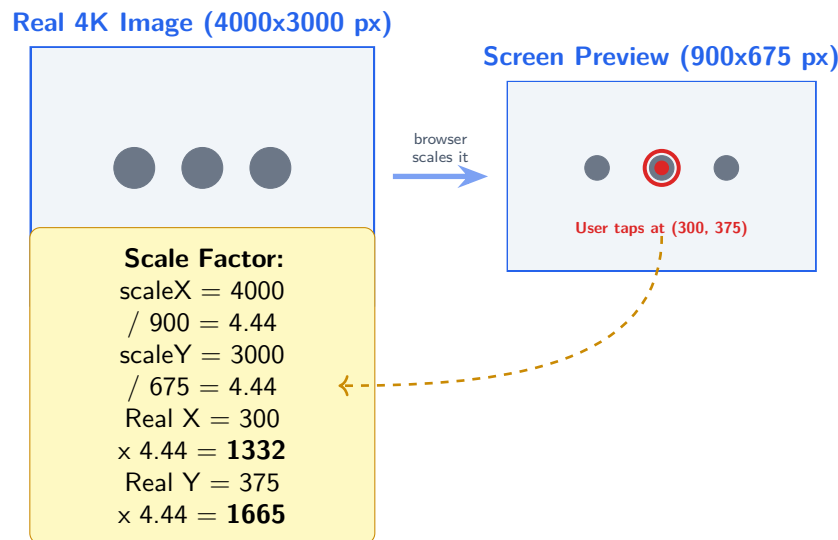
The application guides the user through a linear workflow with numbered steps. Here is how the steps flow:



3.5.3 The Image Click Handler: Scaling Screen Coordinates to Real Image Coordinates

This is one of the most mathematically interesting pieces of the frontend. When a user taps on the screen to mark where a rod is, they are tapping on a *scaled-down preview* of the original high-resolution photograph.

The challenge: if the user taps at screen coordinate (100, 100), we need to figure out what pixel that corresponds to in the full 4K original image.



□ frontend/src/app/app.component.ts -- onImageClick()

```

1 onImageClick(event: MouseEvent) {
2   // Step 1: Get a reference to the actual <img> HTML element.
3   // This is the image element currently visible on the screen.
4   const img = this.imageElement.nativeElement;
5
6   // Step 2: getBoundingClientRect() tells us the size and
7   // position
8   // of the image AS IT APPEARS ON SCREEN (after CSS scaling).
9   // For example, rect.width might be 900px even if
10  // the real image is 4000px.
11  const rect = img.getBoundingClientRect();
12
13  // Step 3: Calculate the scaling ratios.
14  // img.naturalWidth is the REAL width of the original image
15  // file (e.g. 4000).
16  // rect.width is how wide it appears on screen (e.g. 900).
17  // So scaleX = 4000 / 900 = 4.44
18  const scaleX = img.naturalWidth / rect.width;
19  const scaleY = img.naturalHeight / rect.height;
20
21  // Step 4: Apply the ratios to convert click coordinates.
22  // event.clientX is where the user clicked on screen (e.g. 300)
23  // rect.left is how far from the left edge of the screen the
24  // image starts.
25  // (event.clientX - rect.left) gives us position relative to
26  // the image.

```

```

23 // Multiplying by scaleX converts it to the original image
    coordinate.
24 const x = Math.round((event.clientX - rect.left) * scaleX);
25 const y = Math.round((event.clientY - rect.top) * scaleY);
26
27 // Step 5: Store the coordinate in the correct list based on
    current mode.
28 // The spread operator [...this.rodPoints, [x, y]] creates a
    NEW array
29 // (this is important for Angular's change detection to work
    properly).
30 if (this.mode === 'rods') {
31     this.rodPoints = [...this.rodPoints, [x, y]];
32 } else if (this.mode === 'calibration') {
33     this.calibrationPoints = [...this.calibrationPoints, [x, y]];
34 } else if (this.mode === 'reference') {
35     this.referencePoints = [...this.referencePoints, [x, y]];
36 }
37 }

```

Analogy

Imagine looking at a physical map of a city. The map is printed at a scale of 1:1000, meaning 1cm on the map equals 10 meters in real life. If you point to a spot 5cm from the left edge of the map, you know the real building is $5 \times 10 = 50$ meters from that edge. The `onImageClick` function does exactly this, but for pixels instead of centimetres.

3.5.4 Image Upload Handling

When a user selects a photo from their device, the browser provides a `File` object. We need to do two things with it:

1. Show a preview on screen
2. Store it to send to the backend later

 frontend/src/app/app.component.ts -- onImageSelected()

```

1 onImageSelected(event: Event) {
2     // Cast the generic Event to an InputEvent to access files
3     const input = event.target as HTMLInputElement;
4
5     // If no file was selected, exit immediately
6     if (!input.files || !input.files[0]) return;
7
8     // Grab the first file the user selected
9     const file = input.files[0];
10
11     // Store it in our state variable for later use
12     this.uploadedImage = file;
13
14     // Create a FileReader: a browser built-in tool to read file
        contents
15     const reader = new FileReader();
16
17     // This function runs AFTER the file has been fully read into

```

```

    memory
18   reader.onload = (e) => {
19       // e.target.result contains the file data as a data:image/
       jpeg;base64,...
20       // string. Angular can use this string directly as an <img>
       src attribute.
21       this.imagePreviewUrl = e.target?.result as string;
22   };
23
24   // Start reading the file as a Data URL (base64 encoded string)
25   reader.readAsDataURL(file);
26 }

```

❖ What is a FileReader?

FileReader is a built-in browser API. When a user selects a file, the browser gives you a File object, but you cannot directly display or send it. FileReader reads the file's bytes and converts them into either a Base64 Data URL (for display as an image) or an ArrayBuffer (for sending to a server). The onload callback runs when the reading is complete, because file reading is asynchronous.

3.5.5 The Analysis Trigger: Submitting to the Backend

When the user has placed all their markers and clicks “Analyze”, the frontend packages everything up and sends it to the Python backend.

□ frontend/src/app/app.component.ts -- analyzeImage()

```

1  async analyzeImage() {
2      // Show a loading spinner to the user
3      this.isLoading = true;
4
5      // We cannot send a File object over JSON. We must convert it
       to Base64.
6      // This helper function reads the file and returns a base64
       string.
7      const base64Image = await this.fileToBase64(this.uploadedImage
       !);
8
9      // Build the request payload object
10     const payload = {
11         image: base64Image,
12         rod_points: this.rodPoints,
13         calibration_points: this.calibrationPoints,
14         reference_length_mm: this.referenceLength,
15         view_type: this.viewType // "top" or "side"
16     };
17
18     // Send the HTTP POST request to our Python backend
19     // The backend URL comes from environment configuration
20     this.http.post<AnalysisResult>(<
21         `${this.apiUrl}/analyze`,
22         payload
23     ).subscribe({
24         // If the backend responds successfully:

```

```
25     next: (result) => {
26         this.backendResults = result;
27         this.isLoading = false;
28         // Trigger score calculation
29         this.calculateScore();
30     },
31     // If something goes wrong (network error, server error):
32     error: (err) => {
33         console.error('Analysis failed:', err);
34         this.isLoading = false;
35         this.errorMessage = 'Analysis failed. Please try again.';
36     }
37 });
38 }
39
40 // Helper: Convert a File to a Base64 string
41 private fileToBase64(file: File): Promise<string> {
42     return new Promise((resolve, reject) => {
43         const reader = new FileReader();
44         reader.onload = () => {
45             // result is "data:image/jpeg;base64,XXXX..."
46             // We want only the "XXXX..." part after the comma
47             const base64 = (reader.result as string).split(',')[1];
48             resolve(base64);
49         };
50         reader.onerror = reject;
51         reader.readAsDataURL(file);
52     });
53 }
```

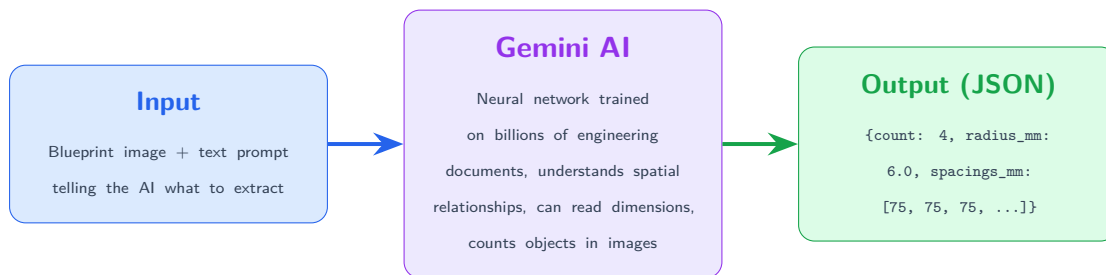
★ async/await and Promises

In JavaScript, operations that take time (like reading files or making network requests) are **asynchronous**: they start and complete in the background while the rest of the code continues. **async/await** is a modern syntax that makes this look like synchronous (step-by-step) code. When you write `await this.fileToBase64(...)`, the code *pauses at that line* until the file is fully read, then continues. Behind the scenes, this is powered by **Promises**: objects that represent a value that will be available in the future.

3.6 The AI Whisperer: `gemini.service.ts`

This service is responsible for the remarkable feat of having a computer read and understand a hand-drawn or CAD-rendered architectural blueprint. It communicates with Google's Gemini Large Language Model, which has been trained on trillions of documents and images.

3.6.1 What is a Large Language Model?



3.6.2 Prompt Engineering: Talking to AI Precisely

The most critical skill when using an AI model in a software application is **Prompt Engineering**: the art of writing instructions so precisely that the AI always returns exactly what you need.

Here is the detailed prompt for analyzing a *top-view* (cross-section) rebar drawing:

❏ frontend/src/app/services/gemini.service.ts -- TOP VIEW PROMPT

```

1  const PROMPT_TOP = '
2  You are a civil engineering drawing analysis expert.
3  Analyze the provided architectural rebar cross-section drawing
4  and extract ONLY the following technical specifications.
5
6  EXTRACTION RULES:
7  1. COUNT: Count all the main round black circles (these represent
8     the cross-sections of steel bars). Do NOT count hatching or
9     filler dots. Only count the solid, clearly defined circles.
10
11  2. RADIUS: Look for dimension labels like "12 dia", "12mm dia",
12     or "T12". The number before "dia" is the diameter.
13     Radius = Diameter / 2. So "12mm dia" = radius of 6mm.
14
15  3. SPACINGS: Generate a list of center-to-center distances
16     between
17     adjacent rods in millimetres.
18     - Follow a strict Clockwise path starting from the Top-Left
19       rod.
19     - If the drawing shows a total width (e.g. "375mm") and the
20       rods appear evenly spaced, compute: spacing = total_width
21       / (number_of_gaps).
21     - Express all values in mm.
22
23  STRICT OUTPUT FORMAT - Return ONLY valid JSON, no other text:
24  {
25    "count": <integer>,
26    "radius_mm": <float>,
27    "spacings_mm": [<float>, <float>, ...]
28  }
29
30  Do NOT include any explanation, markdown, or text outside the
31  JSON.
32  '
  
```

Important Note

Notice how the prompt says “Return ONLY valid JSON, no other text” and “Do NOT include any explanation, markdown, or text”. This is essential. Without these instructions, AI models tend to say things like: *“Sure! Here are the specifications I found: {...}”* – which would break our `JSON.parse()` call, because the text before the `{}` is not valid JSON.

3.6.3 The Side View Prompt

Analyzing the *side view* (elevation) of a column requires completely different instructions, because now we are looking for horizontal stirrups rather than circular cross-sections.

❏ frontend/src/app/services/gemini.service.ts -- SIDE VIEW PROMPT

```

1  const PROMPT_SIDE = `
2  You are a civil engineering drawing analysis expert.
3  Analyze the provided architectural rebar SIDE ELEVATION drawing.
4
5  This is a SIDE VIEW, not a cross-section. You will see:
6  - Vertical lines representing the main longitudinal bars
7  - Horizontal lines (or U-shapes) representing stirrups/ties
8
9  EXTRACTION RULES:
10 1. STIRRUP COUNT: Count the number of horizontal stirrup lines
11   visible in the drawing.
12
13 2. STIRRUP SPACING: Extract the vertical distance between
14   adjacent stirrups in millimetres. If multiple values exist
15   (e.g. closer spacing near column ends), list all of them
16   in order from top to bottom.
17
18 3. BAR DIAMETER: Extract the diameter of the stirrup bars in mm.
19
20 STRICT OUTPUT FORMAT:
21 {
22   "stirrup_count": <integer>,
23   "spacings_mm": [<float>, ...],
24   "bar_diameter_mm": <float>
25 }
26 `;

```

3.6.4 Calling the Gemini API

❏ frontend/src/app/services/gemini.service.ts -- API Call

```

1  async analyzeBlueprint(
2    imageBase64: string,
3    viewType: 'top' | 'side'
4  ): Promise<GeminiResponse> {
5
6    // Choose the right prompt based on what kind of view we have
7    const prompt = viewType === 'top' ? PROMPT_TOP : PROMPT_SIDE;
8
9    // Build the request body in the format Gemini API expects
10   const requestBody = {

```



```
11     contents: [  
12       {  
13         parts: [  
14           // The image part: base64 encoded blueprint photo  
15           {  
16             inline_data: {  
17               mime_type: 'image/jpeg',  
18               data: imageBase64  
19             }  
20           },  
21           // The text part: our detailed prompt  
22           {  
23             text: prompt  
24           }  
25         ]  
26       }  
27     ],  
28     generationConfig: {  
29       temperature: 0,    // 0 = deterministic, always same answer  
30       // for same input  
31       maxOutputTokens: 500 // Keep responses short (just the  
32       JSON)  
33     }  
34   };  
35  
36   // Make the API call  
37   const baseUrl = 'https://gemprism.vercel.app/api/proxy';  
38   const response = await fetch(  
39     `${baseUrl}/gemini-flash-latest:generateContent?key=${this.  
40     apiKey}`,  
41     {  
42       method: 'POST',  
43       headers: { 'Content-Type': 'application/json' },  
44       body: JSON.stringify(requestBody)  
45     }  
46   );  
47  
48   const data = await response.json();  
49  
50   // Extract the text content from Gemini's response  
51   const rawText = data.candidates[0].content.parts[0].text;  
52  
53   // Gemini sometimes wraps JSON in markdown backticks: ```json  
54   // {...} ```  
55   // This regex strips those away to get pure JSON  
56   const cleanJson = rawText.replace(/```json\n?/g, '').replace(/  
57   '\n?/g, '').trim();  
58  
59   // Parse the JSON string into a JavaScript object  
60   return JSON.parse(cleanJson);  
61 }
```

☆ temperature: 0

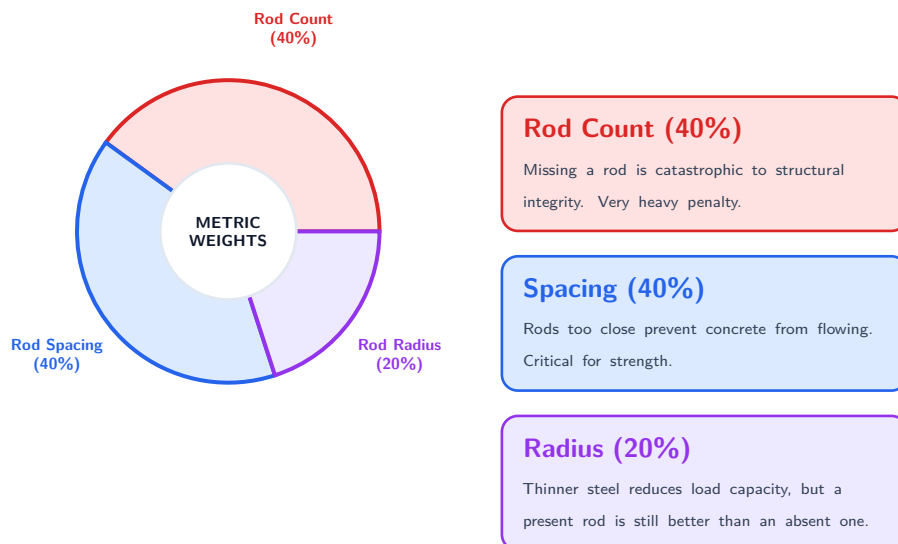
The `temperature` parameter controls how “creative” or “random” the AI’s responses are. A temperature of **1.0** means the AI is very creative and might give different answers each time. A temperature of **0** means the AI is deterministic: given the same input, it always produces the same output. For engineering data extraction, we want determinism, not creativity. We always set temperature to 0.

3.7 The Grader: `scoring.service.ts`

Once we have both the *Target Specifications* (from Gemini reading the blueprint) and the *Actual Measurements* (from the Python backend analyzing the site photo), we need to generate a grade from 0 to 100.

3.7.1 The Scoring Weights

Different types of errors have different levels of consequence in civil engineering. The scoring system reflects this:



3.7.2 The Tolerance Classification

□ `frontend/src/app/services/scoring.service.ts`

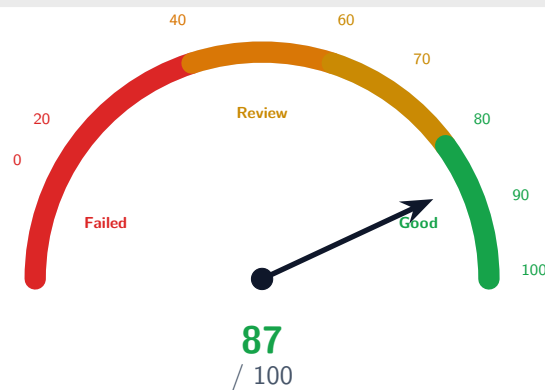
```
1 calculateScore(
2   geminiSpecs: GeminiResponse,
3   backendResults: BackendResult
4 ): ScoringResult {
5
6   const rows: ScoreRow[] = [];
7   let totalWeightedScore = 0;
8
9   // === SECTION 1: ROD COUNT (40% of total score) ===
10  const targetCount = geminiSpecs.count;
11  const actualCount = backendResults.rods.length;
```

```
12
13 // Calculate how wrong the count is, as a percentage
14 const countError = Math.abs(targetCount - actualCount);
15 const countPct = (countError / targetCount) * 100;
16
17 // Classify the error using civil engineering tolerances
18 // Less than 5% error = Acceptable
19 // 5% to 15% = Minor Mismatch (warning)
20 // Over 15% = Not Acceptable (failure)
21 const countStatus = countPct <= 5 ? 'Acceptable' :
22     countPct <= 15 ? 'Minor Mismatch' : 'Not
23     Acceptable';
24
25 // Convert classification to a 0-100 score
26 const countScore = countStatus === 'Acceptable' ? 100 :
27     countStatus === 'Minor Mismatch' ? 60 : 0;
28
29 rows.push({
30     parameter: 'Rod Count',
31     target: targetCount,
32     actual: actualCount,
33     errorPct: countPct,
34     status: countStatus,
35     weight: 0.40
36 });
37
38 totalWeightedScore += countScore * 0.40;
39
40 // === SECTION 2: SPACING ANALYSIS (40% of total score) ===
41 const targetSpacings = geminiSpecs.spacings_mm;
42 const actualSpacings = backendResults.spacings_mm;
43
44 let spacingScoreSum = 0;
45 const spacingCount = Math.min(targetSpacings.length,
46     actualSpacings.length);
47
48 for (let i = 0; i < spacingCount; i++) {
49     const err = Math.abs(targetSpacings[i] - actualSpacings[i]);
50     const pct = (err / targetSpacings[i]) * 100;
51     const status = pct <= 5 ? 'Acceptable' :
52         pct <= 15 ? 'Minor Mismatch' : 'Not Acceptable
53     ';
54     const score = status === 'Acceptable' ? 100 :
55         status === 'Minor Mismatch' ? 60 : 0;
56     spacingScoreSum += score;
57
58     rows.push({
59         parameter: 'Spacing ${i + 1}',
60         target: targetSpacings[i],
61         actual: actualSpacings[i],
62         errorPct: pct,
63         status: status,
64         weight: 0
65     });
66 }
```

```

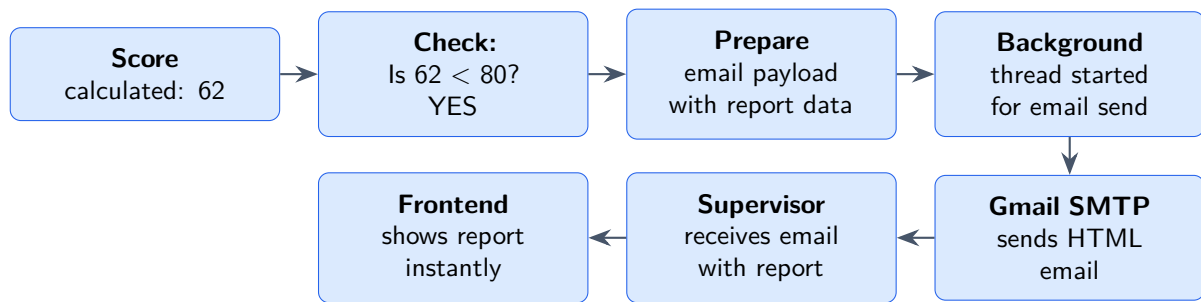
65     const avgSpacingScore = spacingScoreSum / spacingCount;
66     totalWeightedScore += avgSpacingScore * 0.40;
67
68     // === SECTION 3: RADIUS (20% of total score) ===
69     const targetRadius = geminiSpecs.radius_mm;
70     const actualRadius = backendResults.avg_radius_mm;
71     const radiusErr = Math.abs(targetRadius - actualRadius);
72     const radiusPct = (radiusErr / targetRadius) * 100;
73     const radiusStatus = radiusPct <= 5 ? 'Acceptable' :
74                           radiusPct <= 15 ? 'Minor Mismatch' : 'Not
75                           Acceptable';
76     const radiusScore = radiusStatus === 'Acceptable' ? 100 :
77                           radiusStatus === 'Minor Mismatch' ? 60 : 0;
78     totalWeightedScore += radiusScore * 0.20;
79
80     rows.push({
81       parameter: 'Rod Radius',
82       target: targetRadius,
83       actual: actualRadius,
84       errorPct: radiusPct,
85       status: radiusStatus,
86       weight: 0.20
87     });
88
89     // Final score determination
90     const finalScore = Math.round(totalWeightedScore);
91     const overallStatus = finalScore >= 80 ? 'Good' :
92                           finalScore >= 60 ? 'Needs Review' : '
93                           Failed';
94
95     return { finalScore, overallStatus, rows };
96 }

```



3.8 The Alert System: Email Notifications

When the score drops below 80, the system immediately dispatches an HTML email to the project supervisor. This is handled by the backend's email module (called from `app.py`), but the trigger and the recipient details are managed from the frontend configuration.



3.9 The Report Generator

The final step of the frontend workflow generates a beautiful visual report that can be printed or exported as PDF. This is created entirely in HTML and CSS by the Angular component.

The report includes:

- Project header with timestamp and engineer name
- A large circular score badge (green/yellow/red based on result)
- A comparison table: Target vs Actual vs Error% vs Status for every parameter
- Color-coded rows (green for Acceptable, orange for Minor Mismatch, red for Not Acceptable)
- A button to download the Revit highlight JSON file
- A button to print or save as PDF

CHAPTER 4

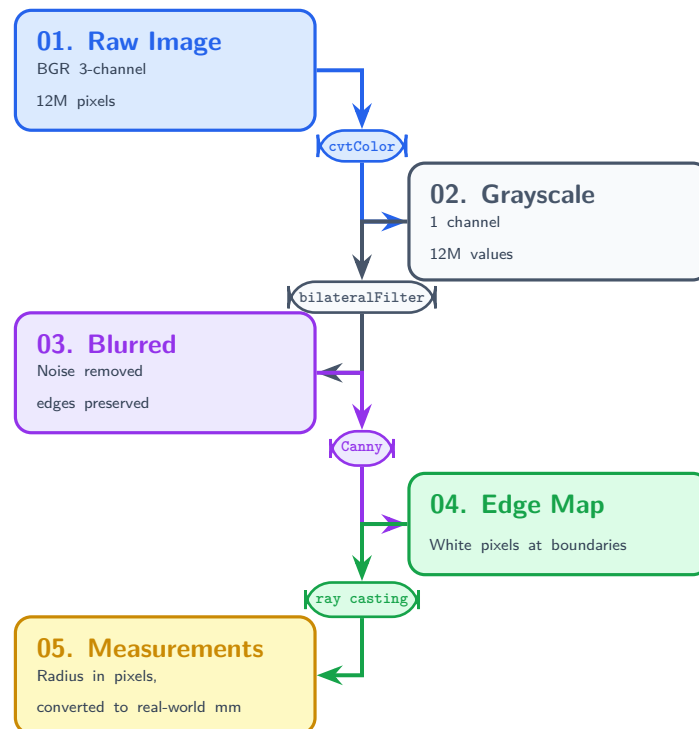
The Backend Brain (Python & OpenCV)

The Backend is where mathematics meets photography. It receives an image file and a set of coordinates, and its job is to precisely measure the steel bars in that image in real-world millimetres. This chapter explains every algorithm, every mathematical function, and every design decision in the Python backend.

4.1 What is OpenCV?

★ OpenCV – Open Source Computer Vision Library

OpenCV is an open-source library originally developed by Intel in 1999. It contains over 2,500 optimised algorithms specifically designed for processing images and video. Under the hood, these algorithms are implemented in highly optimised C++, but Python can access them with simple function calls. When we call `cv2.Canny(image, ...)` in Python, we are invoking thousands of lines of C++ code that run at nearly the speed of machine code.



4.2 The Server Entry: app.py

`app.py` is the Flask server. It listens on a port (typically 5000), receives HTTP requests from the Angular frontend, and routes them to the appropriate service.

❏ backend/src/app.py -- Flask Server Setup

```

1 from flask import Flask, request, jsonify
2 from flask_cors import CORS
3 import base64
4 import numpy as np
5 import cv2
6 import threading
7 from analysis_service import analyze_top_view
8 from side_view_service import analyze_side_view
9 from email_service import send_alert_email
10
11 # Create the Flask application instance
12 app = Flask(__name__)
13
14 # Enable CORS (Cross-Origin Resource Sharing).
15 # This allows the Angular app running on localhost:4200 to make
16 # requests to this server running on localhost:5000.
17 # Without this, browsers block cross-origin requests for security
18
19 CORS(app)
20
21 # Define the analysis endpoint
22 @app.route('/analyze', methods=['POST'])
23 def analyze():

```

```
23     try:
24         # Get the JSON data from the request body
25         data = request.get_json()
26
27         # Extract the base64 image string from the payload
28         image_b64 = data['image']
29
30         # Decode the base64 string back into raw bytes
31         image_bytes = base64.b64decode(image_b64)
32
33         # Convert the raw bytes into a NumPy array (what OpenCV
34         needs)
35         nparr = np.frombuffer(image_bytes, np.uint8)
36
37         # Decode the NumPy array as an image (BGR color format)
38         image = cv2.imdecode(nparr, cv2.IMREAD_COLOR)
39
40         # Extract other parameters from the payload
41         rod_points = data['rod_points']           # [[x1,y1], [
42         x2,y2], ...]
43         calibration_points = data['calibration_points']
44         reference_length_mm = data['reference_length_mm']
45         view_type = data.get('view_type', 'top')
46
47         # Calculate the calibration scale factor (pixels per
48         millimetre)
49         # This converts pixel distances to real-world mm
50         px1, py1 = calibration_points[0]
51         px2, py2 = calibration_points[1]
52         pixel_distance = np.sqrt((px2 - px1)**2 + (py2 - py1)**2)
53         px_per_mm = pixel_distance / reference_length_mm
54
55         # Route to the correct analysis service based on view
56         type
57         if view_type == 'top':
58             results = analyze_top_view(image, rod_points,
59             px_per_mm)
60         else:
61             results = analyze_side_view(image, rod_points,
62             px_per_mm)
63
64         # Return the results as JSON
65         return jsonify(results), 200
66
67     except Exception as e:
68         # Always return a structured error, never crash silently
69         return jsonify({'error': str(e)}), 500
```


4.2.1 CORS: The Security Policy That Must Be Bypassed

Important Note

CORS (Cross-Origin Resource Sharing) is a browser security feature. Browsers refuse to let a web page make HTTP requests to a different domain or port than the one they are running on. For example, a page at localhost:4200 cannot by default make a request to localhost:5000. The `CORS(app)` line on the Flask server adds special HTTP headers to every response, telling the browser “it is okay, the server allows these requests.”

4.2.2 The Email System with Threading

backend/src/app.py -- Email Alert with Threading

```

1 @app.route('/send-alert', methods=['POST'])
2 def alert():
3     data = request.get_json()
4     score = data['score']
5     report_data = data['report_data']
6     recipient_email = data['recipient_email']
7
8     # Only send an email if the score is below the threshold
9     if score < 80:
10         # The email sending can take 2-3 seconds (network round
11         trip to Gmail).
12         # If we do this on the main thread, the user's browser
13         would wait
14         # 2-3 seconds before getting a response. That feels
15         frozen.
16         #
17         # Solution: run send_alert_email in a BACKGROUND THREAD.
18         # The main server thread immediately returns a response,
19         # while the background thread sends the email
20         independently.
21         email_thread = threading.Thread(
22             target=send_alert_email,
23             args=(recipient_email, score, report_data),
24             daemon=True # Thread dies when the main program
25             exits
26         )
27         email_thread.start()
28
29         # Return immediately -- the email is being sent in the
30         background
31         return jsonify({
32             'status': 'alert_queued',
33             'message': 'Email alert has been dispatched.'
34         }), 200
35
36     return jsonify({'status': 'no_alert', 'message': 'Score is
37     acceptable.'}), 200

```

Analogy

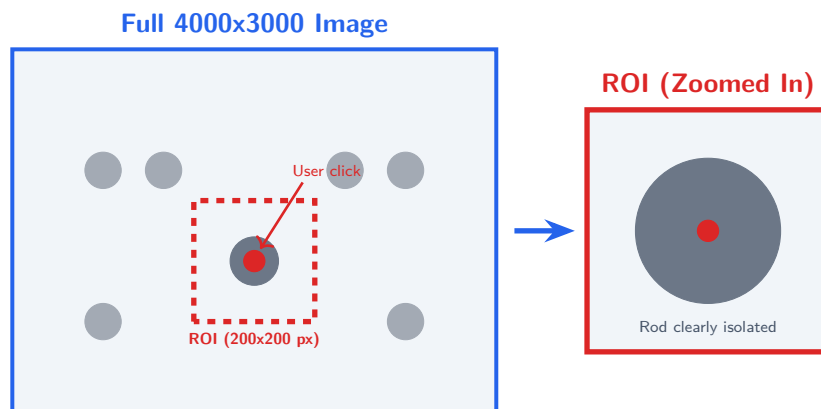
Threading is like ordering food at a restaurant. When you place your order (make the request), the waiter writes it down and immediately walks away (returns a response). The kitchen then prepares your food in parallel (background thread). You do not sit staring at a frozen waiter for 10 minutes. The waiter is already back asking if you want water.

4.3 Computer Vision Core: analysis_service.py

This is the most mathematically intensive file in the entire project. Its mission: given a photograph and a point near the center of a steel rod, find the exact circle that represents that rod's cross-section.

4.3.1 Step 1: Extracting the Region of Interest (ROI)

Processing the entire 4K image for each rod would be wasteful. Instead, we carve out a small square around the user's click point – the **Region of Interest (ROI)**.



❏ backend/src/analysis_service.py -- ROI Extraction

```

1 def find_rod_circle(image: np.ndarray, seed_point: tuple,
2   px_per_mm: float):
3     """
4     Given a full image and a point near the center of a rod,
5     find the exact center and radius of the rod circle.
6
7     Args:
8         image: The full BGR image as a NumPy array
9         seed_point: (x, y) tuple of the user's click coordinate
10        px_per_mm: Calibration factor for mm conversion
11
12    Returns:
13        A dict with center (x,y), radius_px, and radius_mm
14    """
15    x, y = seed_point
16
17    # Define the half-size of the ROI box
18    # We use a box of 200x200 pixels around the click point
19    half_roi = 100

```

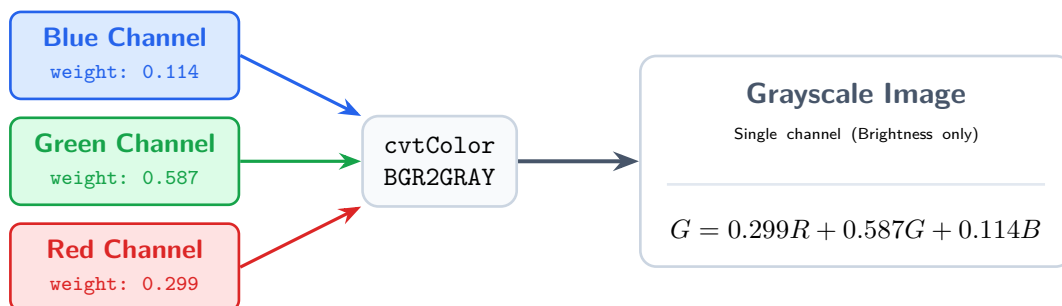
```

19
20     # Calculate ROI boundaries, clamped to image dimensions
21     # max(0, ...) and min(h, ...) prevent going outside the image
22     h, w = image.shape[:2]
23     x_start = max(0, x - half_roi)
24     x_end   = min(w, x + half_roi)
25     y_start = max(0, y - half_roi)
26     y_end   = min(h, y + half_roi)
27
28     # Extract the ROI using NumPy array slicing
29     # image[row_start:row_end, col_start:col_end]
30     roi = image[y_start:y_end, x_start:x_end]
31
32     # Calculate the center of the click point within the ROI
33     # coordinate system
34     # (since ROI starts at x_start, y_start, not 0, 0)
35     center_x = x - x_start
36     center_y = y - y_start
37
38     # Proceed to edge detection and ray casting on the ROI
39     return _detect_circle(roi, center_x, center_y, x, y,
40                          px_per_mm)

```

4.3.2 Step 2: Converting to Grayscale

Color is beautiful to humans, but it is distracting to edge detection algorithms. We have three channels (Red, Green, Blue) but we only need the *brightness* information for edge detection.



The grayscale formula weights the Green channel most heavily (0.587) because the human eye (and cameras) are most sensitive to green light.

❏ backend/src/analysis_service.py -- Preprocessing Pipeline

```

1 def _detect_circle(roi, center_x, center_y, orig_x, orig_y,
2                   px_per_mm):
3
4     # STEP 1: Convert BGR (OpenCV's default) to Grayscale
5     # This reduces the 3-channel (color) image to 1-channel (
6     # brightness)
7     gray = cv2.cvtColor(roi, cv2.COLOR_BGR2GRAY)
8
9     # STEP 2: Bilateral Filter - The Smart Blur
10    # Parameters:
11    # 9 = diameter of pixel neighborhood to consider
12    # 75 = sigmaColor: how much color difference to smooth over

```

```

11     # 75 = sigmaSpace: how far away pixels influence the blur
12     # This filter blurs noise (rust, dirt texture) WHILE
    preserving
13     # the sharp edges of the steel bar boundary. Normal blurs
    destroy edges.
14     blurred = cv2.bilateralFilter(gray, 9, 75, 75)
15
16     # STEP 3: Auto-Calculate Canny Thresholds using Otsu's Method
17     # Otsu's method finds the optimal threshold to separate "
    background"
18     # from "foreground" pixels in the image histogram.
19     # We use this as the upper Canny threshold, and half as the
    lower.
20     high_thresh, _ = cv2.threshold(
21         blurred, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU
22     )
23     low_thresh = 0.5 * high_thresh
24
25     # STEP 4: Canny Edge Detection
26     # This finds the boundaries where pixel brightness changes
    sharply.
27     # The output is a binary image: white (255) at edges, black
    (0) elsewhere.
28     edges = cv2.Canny(blurred, low_thresh, high_thresh)
29
30     # Continue to ray casting...
31     return _ray_cast(edges, center_x, center_y, orig_x, orig_y,
    px_per_mm)

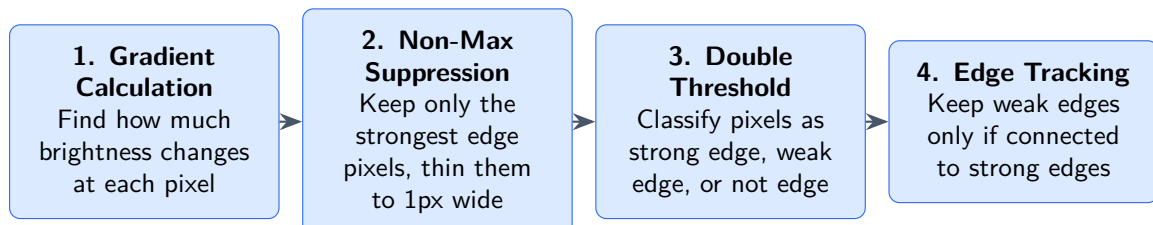
```

☆ Bilateral Filter: The Best of Both Worlds

A **Gaussian blur** smooths everything, including edges. A **Bilateral filter** smooths only areas where pixel values are similar. Near the edge of a steel bar, pixels change dramatically from dark concrete to bright metal. The bilateral filter *recognises* this large change and refuses to blur across it. The result: rust and dirt texture inside the bar is smoothed away, but the sharp steel boundary remains crystal clear for the Canny edge detector.

4.3.3 Step 3: Canny Edge Detection

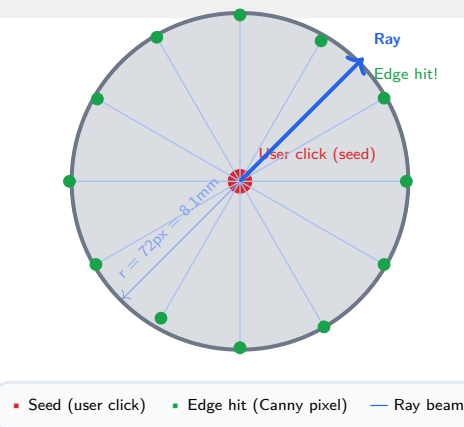
Canny edge detection is a multi-stage algorithm developed by John F. Canny in 1986. It remains the gold standard for edge detection in computer vision.



The output of Canny is a binary image where **white pixels (value 255)** mark the boundaries of objects, and **black pixels (value 0)** are everything else.

4.3.4 Step 4: Ray Casting – The Core Algorithm

This is the most innovative and elegant part of the backend. Ray casting is the algorithm that converts the edge map into a precise measurement of the rod's radius.



□ backend/src/analysis_service.py -- Ray Casting Algorithm

```

1 import numpy as np
2
3 # Pre-compute angles and their cosine/sine values at module load
  time.
4 # This is an optimization: computing cos/sin is expensive. If we
5 # re-computed them for every rod on every frame, it would be slow
6 .
7 # By computing them once globally and reusing them, we save time.
8 NUM_RAYS = 36
9 ANGLES = np.linspace(0, 2 * np.pi, NUM_RAYS, endpoint=False)
10 COS_ANGLES = np.cos(ANGLES)
11 SIN_ANGLES = np.sin(ANGLES)
12
13 def _ray_cast(edges, center_x, center_y, orig_x, orig_y,
14 px_per_mm):
15     """
16     Shoot NUM_RAYS laser beams outward from center_x, center_y
17     and record where each beam hits a white pixel in the edge map
18     .
19     """
20     half_roi = edges.shape[0] // 2 # Maximum ray length
21     radii_found = []
22
23     for i in range(NUM_RAYS):
24         # Get pre-computed direction for this ray
25         cos_a = COS_ANGLES[i]
26         sin_a = SIN_ANGLES[i]
27
28         # Walk outward from center, one pixel step at a time.
29         # We start at radius 4 (to skip the center area where we
30         clicked).
31         for r in range(4, half_roi):
32             # Calculate the pixel coordinate along this ray at

```

```

distance r
29         tx = int(center_x + r * cos_a)
30         ty = int(center_y + r * sin_a)
31
32         # Bounds check: do not read outside the ROI array
33         if tx < 0 or tx >= edges.shape[1] or ty < 0 or ty >=
edges.shape[0]:
34             break
35
36         # If this pixel is white (255) in the edge map, we
found an edge!
37         if edges[ty, tx] > 0:
38             radii_found.append(r)
39             break # Stop walking along this ray
40
41         # Noise filtering: Take only the shortest 60% of radii.
42         # Why? Some rays might hit spurious edges inside the rod (
43         rust marks,
44         # shadow lines) before reaching the true boundary. These give
45         artificially
46         # small radii. But the TRUE boundary gives mostly consistent
47         radii,
48         # while noise is random. Taking the bottom 60% filters out
49         rays that
50         # shot through and hit background noise beyond the true edge.
51         radii_found.sort()
52         valid_count = int(len(radii_found) * 0.6)
53         valid_radii = radii_found[:valid_count]
54
55         if not valid_radii:
56             # Fallback: if no edges were found, estimate from typical
57             bar size
58             radius_px = 25.0
59         else:
60             # Use the MEDIAN, not the mean.
61             # Median is more robust: a single outlier does not skew
62             it.
63             radius_px = float(np.median(valid_radii))
64
65         # Convert pixels to millimetres using the calibration factor
66         radius_mm = radius_px / px_per_mm
67
68         return {
69             'center': [orig_x, orig_y],
70             'radius_px': round(radius_px, 2),
71             'radius_mm': round(radius_mm, 2)
72         }

```

Analogy

You are standing in a dark, perfectly round room. You have a laser pointer. You point it straight ahead, walk forward until the red dot hits the wall, and note the distance: 3 metres. You turn 10 degrees and repeat. After 36 measurements, you sort them: most say 2.9-3.1 metres (the true wall distance). A few outliers say 0.5 metres (you accidentally hit a piece

of furniture). You discard those small outliers and take the median of the rest: 3.0 metres. That is the room's radius. Precisely what the code above does.

4.3.5 Calculating Spacing Between Rods

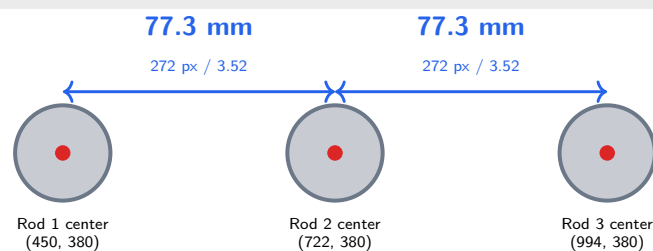
Once every rod's center and radius are known, calculating spacing is straightforward:

❏ backend/src/analysis_service.py -- Spacing Calculation

```

1 def calculate_spacings(rods: list, px_per_mm: float) -> list:
2     """
3     Calculate centre-to-centre spacing between adjacent rods.
4
5     Note: "spacing" in civil engineering means the gap between
6     the
7     EDGES of bars, but some specifications mean centre-to-centre.
8     This implementation returns centre-to-centre distance in mm,
9     which matches how most architectural drawings specify it.
10    """
11    spacings_mm = []
12
13    for i in range(len(rods) - 1):
14        # Get the centre coordinates of consecutive rods
15        x1, y1 = rods[i]['center']
16        x2, y2 = rods[i + 1]['center']
17
18        # Euclidean distance: sqrt( (x2-x1)^2 + (y2-y1)^2 )
19        pixel_distance = np.sqrt((x2 - x1)**2 + (y2 - y1)**2)
20
21        # Convert pixels to millimetres
22        distance_mm = pixel_distance / px_per_mm
23
24        spacings_mm.append(round(distance_mm, 1))
25
26    return spacings_mm

```



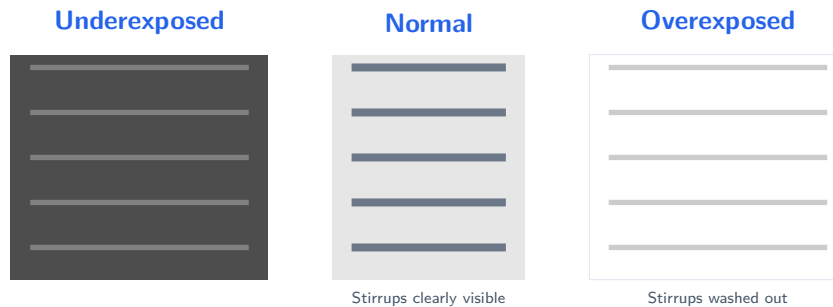
4.4 Side Elevation Analysis: side_view_service.py

Analyzing the side of a column is fundamentally different from the top view. From the side, we are looking for horizontal stirrups – bands of steel that wrap around the vertical bars.

4.4.1 The Challenge: Variable Lighting

Construction site photographs are rarely taken under controlled lighting. A side view photo might be:

- Overexposed (too bright) due to sunlight behind the camera
- Underexposed (too dark) due to the column being in shadow
- Uneven, with one side bright and the other dark



4.4.2 CLAHE: Adaptive Contrast Enhancement

The solution is **CLAHE** (Contrast Limited Adaptive Histogram Equalization). Unlike standard contrast enhancement that applies one adjustment to the whole image, CLAHE works on small tiles independently.

❏ backend/src/side_view_service.py -- CLAHE Preprocessing

```
1 def preprocess_side_image(image: np.ndarray) -> np.ndarray:
2     """
3     Enhance the contrast of a side-view image to make stirrups
4     visible regardless of lighting conditions.
5     """
6     # Convert to grayscale
7     gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
8
9     # Create a CLAHE object
10    # clipLimit=2.0: limits the maximum contrast enhancement (
11    # prevents
12    # amplifying noise in very uniform regions)
13    # tileGridSize=(8,8): divides the image into 8x8 = 64 tiles
14    # and
15    # applies histogram equalization independently to each tile
16    .
17    clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
18
19    # Apply CLAHE to the grayscale image
20    enhanced = clahe.apply(gray)
21
22    return enhanced
```

4.4.3 Morphological Operations: Finding Horizontal Lines

After preprocessing, we use **morphological operations** to isolate the horizontal stirrups from the rest of the image.

1. Binary Edge Image

2. After Morphological Opening

3. Detected Stirrups


☐ backend/src/side_view_service.py -- Morphological Line Detection

```

1 def detect_stirrups(preprocessed: np.ndarray, px_per_mm: float)
  -> dict:
2     """
3     Find the positions of horizontal stirrups in a preprocessed
  image.
4     Uses morphological opening to isolate horizontal lines.
5     """
6     h, w = preprocessed.shape
7
8     # STEP 1: Threshold the image to get binary (black/white)
9     # THRESH_OTSU automatically finds the best threshold value
10    _, binary = cv2.threshold(
11        preprocessed, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU
12    )
13
14    # STEP 2: Create a horizontal kernel
15    # A kernel is a small matrix used in morphological operations
16    .
17    # This kernel is 1 pixel tall and 40 pixels wide.
18    # When we "open" the image with this kernel, only features
  that
19    # are AT LEAST 40 pixels wide horizontally survive.
20    # Stirrups are long horizontal bars -- they survive.
21    # Dots, vertical lines, and small noise get removed.
22    horizontal_kernel_length = max(20, w // 20)
23    horizontal_kernel = cv2.getStructuringElement(
24        cv2.MORPH_RECT,
25        (horizontal_kernel_length, 1) # (width, height) of
  kernel
26    )
27
28    # STEP 3: Apply morphological opening
29    # "Opening" = erosion followed by dilation.
30    # Erosion shrinks all white regions (removes thin/small
  features).
31    # Dilation then expands the remaining (only large horizontal)
  features back.
32    horizontal_lines = cv2.morphologyEx(
33        binary, cv2.MORPH_OPEN, horizontal_kernel
34    )
35
36    # STEP 4: Find the Y-coordinate (vertical position) of each
  stirrup line

```

```

36     # We project the binary image horizontally (sum all columns
37     # for each row).
38     # Rows with many white pixels (stirrup rows) will have high
39     # sums.
40     row_sums = np.sum(horizontal_lines, axis=1)
41
42     # Find peaks in the row_sums: these are the Y-coordinates of
43     # stirrups
44     stirrup_ys = _find_peaks(row_sums, threshold=w * 0.3)
45
46     # STEP 5: Calculate spacings between stirrups
47     spacings_mm = []
48     for i in range(1, len(stirrup_ys)):
49         pixel_gap = stirrup_ys[i] - stirrup_ys[i-1]
50         spacings_mm.append(round(pixel_gap / px_per_mm, 1))
51
52     return {
53         'stirrup_count': len(stirrup_ys),
54         'stirrup_positions_px': stirrup_ys.tolist(),
55         'spacings_mm': spacings_mm,
56         'avg_spacing_mm': round(np.mean(spacings_mm), 1) if
57         spacings_mm else 0
58     }

```

4.4.4 Line Fitting with Polynomial Regression

When multiple points are detected along a stirrup (rather than just one clean line), we use **linear regression** to find the best-fit line through them. This gives us a robust estimate of the stirrup's Y-position even if some points are slightly off due to image noise.

❏ backend/src/side_view_service.py -- Line Fitting

```

1  import numpy as np
2
3  def fit_horizontal_line(points: list) -> float:
4      """
5      Given a list of (x, y) points that should all lie on the same
6      horizontal line, find the best-fit Y-coordinate.
7
8      We use np.polyfit with degree=1 (linear) to fit y = mx + c.
9      For a truly horizontal line, m should be near 0 and c gives
10     us Y.
11     """
12     if len(points) < 2:
13         return points[0][1] if points else 0
14
15     xs = np.array([p[0] for p in points], dtype=float)
16     ys = np.array([p[1] for p in points], dtype=float)
17
18     # polyfit(x, y, degree) returns coefficients [m, c] for y =
19     # mx + c
20     # For our horizontal lines, we care mostly about c (the Y
21     # intercept)
22     m, c = np.polyfit(xs, ys, 1)

```

```

21     # Evaluate the line at the center x-position of the image
22     # to get the canonical Y-coordinate for this stirrup
23     center_x = np.mean(xs)
24     best_y = m * center_x + c
25
26     return round(best_y, 1)

```

★ Why np.polyfit instead of just averaging Y values?

If all the detected points for a stirrup had exactly the same Y value, we could just average them. But due to image noise, a single stirrup might generate edge points at Y=380, Y=382, Y=379, Y=385. Simply averaging gives 381.5, but some outlier at 385 could skew it. polyfit fits the best mathematical line through all points, which is more robust to outliers.

4.5 The Email Service: email_service.py

When the quality score drops below 80%, an automated HTML email is dispatched to the project supervisor. This email includes the complete analysis report.

❏ backend/src/email_service.py

```

1  import smtplib
2  from email.mime.multipart import MIMEMultipart
3  from email.mime.text import MIMEText
4  import os
5
6  def send_alert_email(recipient: str, score: float, report_data:
    dict):
7      """
8      Send an HTML alert email via Gmail SMTP.
9      This function is designed to run in a background thread.
10     """
11     # Retrieve credentials from environment variables
12     # (NEVER hardcode credentials in source code!)
13     sender_email = os.environ.get('EMAIL_USER')
14     sender_password = os.environ.get('EMAIL_PASSWORD')
15
16     # Build the email container
17     msg = MIMEMultipart('alternative')
18     msg['Subject'] = f'[ALERT] Rebar QC Score: {score}/100 -
    Immediate Review Required'
19     msg['From'] = sender_email
20     msg['To'] = recipient
21
22     # Create the HTML body
23     # We use an f-string to inject the actual score and data
24     dynamically
25     html_body = f"""
26     <html>
27     <body style="font-family: Arial, sans-serif; background-color
    : #f8f9fa;">

```

```

27     <div style="max-width: 600px; margin: 0 auto; padding: 20
28 px;">
29         <!-- Alert Header -->
30         <div style="background-color: #dc2626; color: white;
31 padding: 20px;
32                 border-radius: 8px; text-align: center;">
33             <h1>Quality Control Alert</h1>
34             <h2>Score: {score}/100</h2>
35         </div>
36         <!-- Score Summary -->
37         <div style="background-color: white; padding: 20px;
38 margin-top: 20px;
39                 border-radius: 8px; border: 1px solid #
40 e2e8f0;">
41             <p>The automated rebar inspection has detected a
42 quality score
43             below the acceptable threshold of 80/100.</p>
44             <p><strong>Immediate site inspection is
45 recommended.</strong></p>
46         </div>
47         <!-- Report Table -->
48         <table style="width: 100%; border-collapse: collapse;
49 margin-top: 20px;">
50             <tr style="background-color: #2563eb; color:
51 white;">
52                 <th style="padding: 12px;">Parameter</th>
53                 <th style="padding: 12px;">Target</th>
54                 <th style="padding: 12px;">Actual</th>
55                 <th style="padding: 12px;">Status</th>
56             </tr>
57             """
58             # Add a table row for each parameter in the report
59             for row in report_data.get('rows', []):
60                 if row['status'] == 'Acceptable':
61                     status_color = '#22c55e'
62                 elif row['status'] == 'Minor Mismatch':
63                     status_color = '#f59e0b'
64                 else:
65                     status_color = '#ef4444'
66
67                 html_body += f"""
68                 <tr>
69                     <td style="padding: 10px; border: 1px solid #
70 e2e8f0;">{row['parameter']}</td>
71                     <td style="padding: 10px; border: 1px solid #
72 e2e8f0;">{row['target']}</td>
73                     <td style="padding: 10px; border: 1px solid #
74 e2e8f0;">{row['actual']}</td>
75                     <td style="padding: 10px; border: 1px solid #
76 e2e8f0;
77                             color: {status_color}; font-weight
78 : bold;">{row['status']}</td>

```

```
70         </tr>
71     """
72
73     html_body += """
74         </table>
75     </div>
76 </body>
77 </html>
78 """
79
80 # Attach the HTML to the email
81 part = MIMEText(html_body, 'html')
82 msg.attach(part)
83
84 # Connect to Gmail's SMTP server and send
85 # Port 587 uses STARTTLS encryption
86 try:
87     with smtplib.SMTP('smtp.gmail.com', 587) as server:
88         server.ehlo()                # Identify ourselves to
the server
89         server.starttls()            # Upgrade connection to
encrypted TLS
90         server.login(sender_email, sender_password)
91         server.sendmail(sender_email, recipient, msg.
as_string())
92     except Exception as e:
93         print(f"Email sending failed: {e}")
94         # We intentionally do not re-raise: email failure should
not
95         # crash the analysis workflow. The result is still
returned.
```

Important Note

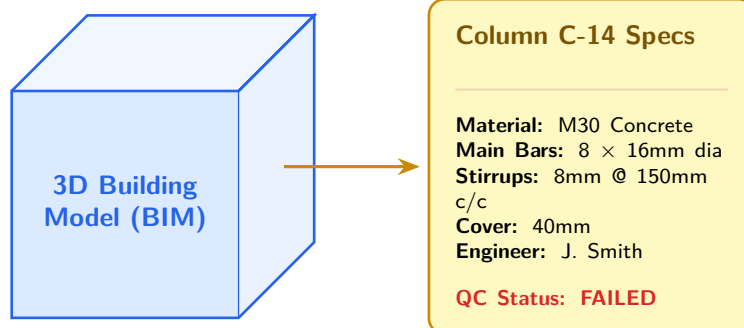
Notice `os.environ.get('EMAIL_USER')` and `os.environ.get('EMAIL_PASSWORD')`. **Never** hardcode email credentials (or any secrets) directly in source code. If that code is ever committed to GitHub (even accidentally), the credentials are exposed to the entire world. Environment variables are a standard secure way to store secrets: they live in the server's operating system, not in the code files.

CHAPTER 5

BIM Integration (Autodesk Revit)

A quality control report is valuable. But showing the defect directly in a 3D model of the building is transformative. This chapter explains how the RebarAnalysis web platform connects to Autodesk Revit, the industry-standard software for 3D Building Information Modelling (BIM), and how it highlights defective rebar elements with red visual markers.

5.1 What is Building Information Modelling?

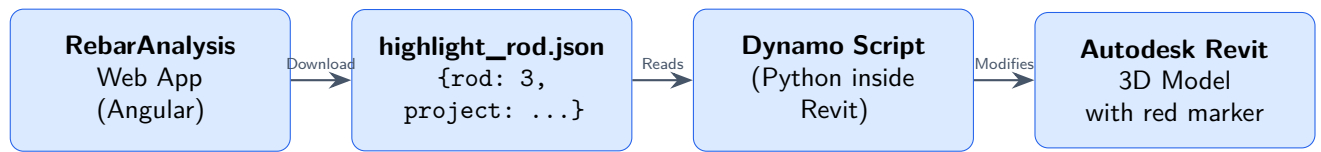


★ BIM vs a Regular 3D Model

A regular 3D model in software like Blender or SketchUp is just a visual representation: pretty shapes with no knowledge of what they are. A **BIM model** in Autodesk Revit is intelligent: every wall, column, beam, and piece of rebar is an *object* with associated data (material, dimensions, cost, engineering specifications, quality status). RebarAnalysis writes directly to this data layer.

5.2 Dynamo: The Bridge Between Python and Revit

Dynamo is a visual programming environment that runs inside Autodesk Revit. It allows engineers to write scripts (including Python scripts) that interact directly with the Revit model database.



5.3 The Highlight JSON File

When the frontend detects a defective rod, the user can download a tiny JSON file that encodes exactly which rod needs to be highlighted.

□ highlight_rod.json -- Example File

```

1 {
2   "rod": 3,
3   "project_id": "COL-C14-BLOCK-A",
4   "column_id": "C14",
5   "issue_type": "spacing_mismatch",
6   "severity": "Not Acceptable",
7   "actual_spacing_mm": 112.4,
8   "target_spacing_mm": 75.0,
9   "error_pct": 49.9,
10  "generated_at": "2024-11-28T14:32:00Z"
11 }
  
```

5.4 The Revit Highlight Script: rodhighlight.py

This Python script runs inside Dynamo inside Revit. It reads the JSON file, finds the specified rebar element in the 3D model, and applies a red visual override.

□ test_dataset/revit_models/dynamo_scripts/rodhighlight.py -- Full Script

```

1 import json
2 import os
3 import math
4
5 # These imports come from Revit's own API
6 from Autodesk.Revit.DB import (
7     FilteredElementCollector, Rebar, Color,
8     OverrideGraphicSettings, XYZ, Plane, Arc, Line,
9     Transaction, CurveLoop, ModelCurveArray
10 )
11
12 # Dynamo gives us access to the document (the open Revit model)
13 # and the active view (what is currently visible on screen)
14 doc = DocumentManager.Instance.CurrentDBDocument
15 active_view = doc.ActiveView
16
17 # =====
  
```

```

18 # STEP 1: Read the JSON file from the user's Downloads folder
19 # =====
20 json_path = os.path.join(
21     os.path.expanduser("~"), "Downloads", "highlight_rod.json"
22 )
23
24 if not os.path.exists(json_path):
25     raise FileNotFoundError(
26         f"Could not find highlight_rod.json at: {json_path}\n"
27         "Please download the file from RebarAnalysis first."
28     )
29
30 with open(json_path, "r") as f:
31     data = json.load(f)
32
33 TARGET_ROD_INDEX = data.get("rod", None)
34 COLUMN_ID = data.get("column_id", None)
35
36 if TARGET_ROD_INDEX is None:
37     raise ValueError("JSON file must contain a 'rod' field (
38         integer index).")
39 # =====
40 # STEP 2: Collect all Rebar elements from the 3D model
41 # =====
42 # FilteredElementCollector is Revit's query language.
43 # It lets us search the model database by element type, parameter
44 # , etc.
45 rebar_elements = list(
46     FilteredElementCollector(doc)
47     .OfClass(Rebar)
48     .ToElements()
49 )
50 # Filter to only the rebar in the specified column (if column_id
51 # given)
52 if COLUMN_ID:
53     column_rebar = [
54         r for r in rebar_elements
55         if r.LookupParameter("Host Mark") and
56         r.LookupParameter("Host Mark").AsString() == COLUMN_ID
57     ]
58 else:
59     column_rebar = rebar_elements
60
61 if TARGET_ROD_INDEX >= len(column_rebar):
62     raise IndexError(
63         f"Rod index {TARGET_ROD_INDEX} is out of range. "
64         f"Only {len(column_rebar)} rebar elements found."
65     )
66
67 # Get the specific defective rebar object
68 target_rebar = column_rebar[TARGET_ROD_INDEX]
69 # =====
70 # STEP 3: Get the 3D location of the target rebar

```



```

71 # =====
72 # Rebar elements have a GetCenterlineCurves() method that returns
73 # the 3D curves defining the rebar's path through space.
74 curves = target_rebar.GetCenterlineCurves(False, False, False)
75 if not curves:
76     raise ValueError("Could not get curves for the target rebar
77         element.")
78
79 # Get the midpoint of the first curve as our highlight center
80 midpoint = curves[0].Evaluate(0.5, True)
81 cx, cy, cz = midpoint.X, midpoint.Y, midpoint.Z
82
83 # =====
84 # STEP 4: Draw a red highlight circle around the defective rod
85 # =====
86 # All model modifications in Revit must happen inside a
87 # Transaction.
88 # Think of a Transaction like a database transaction: it either
89 # completes fully, or rolls back completely if anything fails.
90 t = Transaction(doc, "Highlight Defective Rebar")
91 t.Start()
92
93 try:
94     # Create a horizontal plane at the rod's location
95     origin = XYZ(cx, cy, cz)
96     x_axis = XYZ(1, 0, 0)
97     y_axis = XYZ(0, 1, 0)
98     plane = Plane.CreateByOriginAndBasis(origin, x_axis, y_axis)
99
100     # Create a circle of radius 150mm (in Revit's internal units:
101     # feet)
102     # Revit uses feet internally; 150mm = 0.4921 feet
103     r = 0.4921
104
105     # An Arc in Revit is created as two half-circles (0 to pi, pi
106     # to 2pi)
107     arc1 = Arc.Create(plane, r, 0, math.pi)
108     arc2 = Arc.Create(plane, r, math.pi, 2 * math.pi)
109
110     # Place the arcs as model curves in the 3D view
111     mc1 = doc.Create.NewModelCurve(arc1, active_view.SketchPlane)
112     mc2 = doc.Create.NewModelCurve(arc2, active_view.SketchPlane)
113
114     #
115     =====
116     # STEP 5: Apply the red visual override
117     #
118     =====
119     # Create an override settings object
120     override = OverrideGraphicSettings()
121
122     # Set the line color to pure red (RGB: 255, 0, 0)
123     red_color = Color(255, 0, 0)
124     override.SetProjectionLineColor(red_color)
125
126     # Set the line weight to 6 (very thick, highly visible)

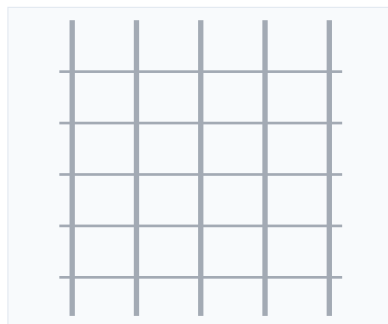
```

```

121     override.SetProjectionLineWeight(6)
122
123     # Apply the override to both arcs in the active view
124     active_view.SetElementOverrides(mc1.Id, override)
125     active_view.SetElementOverrides(mc2.Id, override)
126
127     # Also override the rebar element itself
128     override_rebar = OverrideGraphicSettings()
129     override_rebar.SetProjectionLineColor(Color(255, 0, 0))
130     override_rebar.SetProjectionLineWeight(5)
131     active_view.SetElementOverrides(target_rebar.Id,
132     override_rebar)
133
134     # Commit the transaction
135     t.Commit()
136     print(f"Successfully highlighted Rod {TARGET_ROD_INDEX} in
137     column {COLUMN_ID}")
138
139 except Exception as e:
140     # If anything went wrong, roll back ALL changes
141     t.Rollback()
142     raise e

```

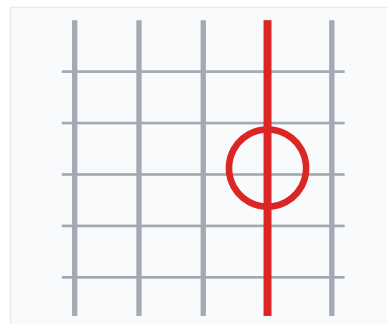
Revit 3D View (Before)



All bars look the same

Run
Dynamo
Script

Revit 3D View (After)



Rod #3: DEFECTIVE

✓ Technical Insight

The power of this integration is remote supervision. A project manager in Mumbai can look at the 3D Revit model of a building being constructed in Pune. The site engineer runs the RebarAnalysis app, downloads the JSON, and sends it to the manager. The manager runs the Dynamo script and instantly sees a bright red circle around the exact piece of steel that was installed incorrectly – without ever setting foot on the site.

5.5 The Data Export Script: write_report.py

Beyond just highlighting, there is a companion script that writes the full analysis data *into* the Revit model's database, so the information is permanently stored with the element.

□ test_dataset/revit_models/dynamo_scripts/write_report.py

```

1 import json
2 import os
3 from Autodesk.Revit.DB import FilteredElementCollector, Rebar,
  Transaction
4
5 doc = DocumentManager.Instance.CurrentDBDocument
6
7 # Load the complete report JSON (not just the highlight JSON)
8 report_path = os.path.join(
9     os.path.expanduser("~"), "Downloads", "rebaranalysis_report.
10     json"
11 )
12 with open(report_path, "r") as f:
13     report = json.load(f)
14
15 column_id = report['column_id']
16 score = report['final_score']
17 status = report['overall_status']
18
19 # Get all rebar in the specified column
20 rebar_elements = list(
21     FilteredElementCollector(doc).OfClass(Rebar).ToElements()
22 )
23
24 t = Transaction(doc, "Write RebarAnalysis Report to BIM")
25 t.Start()
26
27 try:
28     for rebar in rebar_elements:
29         host_param = rebar.LookupParameter("Host Mark")
30         if host_param and host_param.AsString() == column_id:
31
32             # Write the QC score to a Revit shared parameter
33             # This data is now permanently stored in the BIM
34             model
35
36             score_param = rebar.LookupParameter("QC_Score")
37             if score_param:
38                 score_param.Set(str(score))
39
40             status_param = rebar.LookupParameter("QC_Status")
41             if status_param:
42                 status_param.Set(status)
43
44             date_param = rebar.LookupParameter("QC_Date")
45             if date_param:
46                 date_param.Set(report.get('generated_at', '
47                 Unknown'))
48
49         t.Commit()
50         print(f"Report written to BIM for column {column_id}")
51
52 except Exception as e:
53     t.Rollback()
54     raise e

```

6

CHAPTER

Execution and Deployment

Writing beautiful code is only half the job. The other half is making that code actually *run* reliably. This chapter covers how to start the system locally for development, and how it is deployed to the cloud for real-world use.

6.1 The Launcher: `run.bat`

For local development on Windows, a Batch script automates the entire setup. Running this single file starts both the Frontend and Backend, installs dependencies, and opens the browser.

❏ `run.bat` -- Windows Launch Script

```
1 @echo off
2 title RebarAnalysis - System Launcher
3 color 0B
4
5 echo =====
6 echo RebarAnalysis - Starting All Services
7 echo =====
8 echo.
9
10 :: Check if Node.js is installed
11 where node >nul 2>nul
12 if %ERRORLEVEL% NEQ 0 (
13     echo ERROR: Node.js is not installed!
14     echo Please download it from https://nodejs.org
15     pause
16     exit /b 1
17 )
18
19 :: Check if Python is installed
20 where python >nul 2>nul
21 if %ERRORLEVEL% NEQ 0 (
22     echo ERROR: Python is not installed!
23     echo Please download it from https://python.org
24     pause
```

```
25     exit /b 1
26 )
27
28 :: Prompt for credentials if not already set
29 if not defined GEMINI_API_KEY (
30     set /p GEMINI_API_KEY=Enter your Gemini API Key:
31 )
32 if not defined EMAIL_USER (
33     set /p EMAIL_USER=Enter your Gmail address:
34 )
35 if not defined EMAIL_PASSWORD (
36     set /p EMAIL_PASSWORD=Enter your Gmail App Password:
37 )
38
39 :: Export as environment variables for child processes
40 set GEMINI_API_KEY=%GEMINI_API_KEY%
41 set EMAIL_USER=%EMAIL_USER%
42 set EMAIL_PASSWORD=%EMAIL_PASSWORD%
43
44 echo.
45 echo Installing Python dependencies...
46 cd backend
47 pip install -r requirements.txt --quiet
48 cd ..
49
50 echo Installing Angular dependencies...
51 cd frontend
52 call npm install --silent
53 cd ..
54
55 echo.
56 echo Starting Python Backend on port 5000...
57 start "RebarAnalysis Backend" cmd /c "cd backend && python src/
    app.py"
58
59 echo Starting Angular Frontend on port 4200...
60 start "RebarAnalysis Frontend" cmd /c "cd frontend && npm start"
61
62 :: Wait 5 seconds for servers to boot up
63 timeout /t 5 /nobreak >nul
64
65 echo.
66 echo Opening browser...
67 start http://localhost:4200
68
69 echo.
70 echo =====
71 echo Both services are now running!
72 echo Frontend: http://localhost:4200
73 echo Backend: http://localhost:5000
74 echo =====
75 echo.
76 echo Close the server windows to stop all services.
77 pause
```

6.1.1 Understanding the Batch Script

- `@echo off`: Prevents the script from printing every command as it executes. Only the explicit echo statements are shown.
- `where node`: Checks if node is accessible via the system PATH. If not, ERRORLEVEL is set to 1.
- `set /p`: Prompts the user to type input and stores it in a variable.
- `start "Title" cmd /c "..."`: Opens a new Command Prompt window with the given title and runs the command inside it. This is how we run two servers simultaneously – each in its own window.
- `timeout /t 5`: Waits 5 seconds before continuing. This gives the servers time to fully boot before the browser is opened.

6.2 The Backend Dependencies: requirements.txt

❏ backend/requirements.txt

```

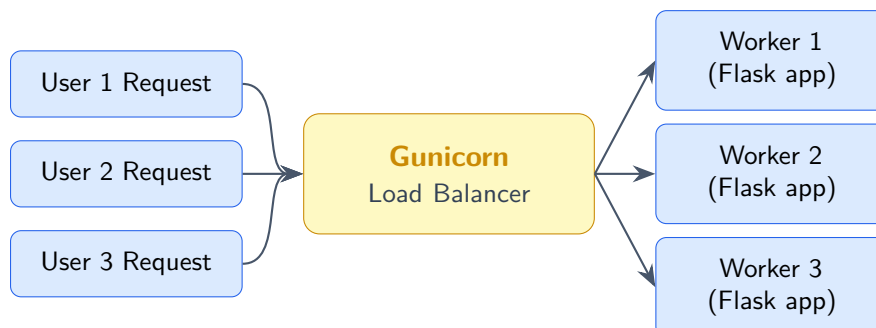
1 flask==3.0.0           # Web server framework
2 flask-cors==4.0.0      # Cross-Origin Resource Sharing support
3 numpy==1.26.2          # Numerical array operations
4 opencv-python==4.8.1.78 # Computer vision algorithms
5 Pillow==10.1.0         # Image format handling (PNG, JPEG, etc.)
6 gunicorn==21.2.0       # Production WSGI server (for Render
   deployment)
7 python-dotenv==1.0.0    # Load environment variables from .env
   files

```

6.2.1 Why Gunicorn in Production?

Flask's built-in server (used when you run `python app.py`) is a *development server*. It is single-threaded: it handles one request at a time. If two users upload images simultaneously, the second user waits for the first to finish.

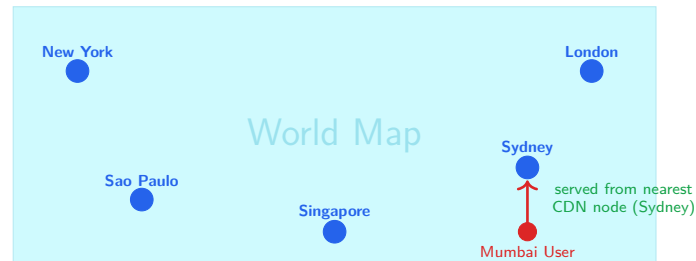
Gunicorn (Green Unicorn) is a production-grade WSGI server. It starts multiple worker processes, each capable of handling requests independently.



6.3 Cloud Deployment Architecture

6.3.1 Vercel: Hosting the Angular Frontend

Vercel is a cloud platform specialising in hosting frontend JavaScript applications. When code is pushed to the GitHub repository, Vercel automatically builds the Angular app (`ng build -prod`) and deploys it to its global Content Delivery Network (CDN).



A CDN stores copies of the website at servers around the world. When a user in Mumbai visits the site, they are served from a nearby CDN node (Singapore or Sydney) instead of a central server, resulting in significantly lower latency.

6.3.2 Render: Hosting the Python Backend

Render is a cloud platform for hosting backend services. Unlike the static Angular files, the Python backend needs a running server process with sufficient CPU power to handle OpenCV computations.

render.yaml -- Render Deployment Configuration

```
1 services:
2   - type: web
3     name: rebaranalysis-backend
4     env: python
5     region: singapore # Choose closest region to your users
6     plan: standard    # Provides 1 CPU, 2GB RAM
7     buildCommand: pip install -r requirements.txt
8     startCommand: gunicorn -w 4 -b 0.0.0.0:10000 src.app:app
9     envVars:
10      - key: GEMINI_API_KEY
11        sync: false # Must be manually set in Render dashboard
12      - key: EMAIL_USER
13        sync: false
14      - key: EMAIL_PASSWORD
15        sync: false
```

The startCommand breaks down as:

- **gunicorn**: Use Gunicorn instead of Flask's dev server
- **-w 4**: Start 4 worker processes (handles 4 concurrent requests)
- **-b 0.0.0.0:10000**: Bind to all interfaces on port 10000
- **src.app:app**: Import the Flask app object from src/app.py

6.4 Environment Configuration: `environment.ts`

Angular has separate environment files for development and production. This allows the app to automatically point to the correct backend URL.

❏ frontend/src/environments/environment.ts -- Development

```
1 export const environment = {
2   production: false,
3   // During development, the backend runs locally
4   apiUrl: 'http://localhost:5000',
5   // Gemini API key (loaded from env or entered at runtime)
6   geminiApiKey: process.env['GEMINI_API_KEY'] || ''
7 };
```

❏ frontend/src/environments/environment.prod.ts -- Production

```
1 export const environment = {
2   production: true,
3   // In production, use the Render cloud backend URL
4   apiUrl: 'https://rebaranalysis-backend.onrender.com',
5   geminiApiKey: '' // Loaded from Vercel environment variables
6 };
```

6.5 The Complete File Structure

Here is the complete directory structure of the entire project:

❏ RebarAnalysis -- Project Directory Tree

```
1 RebarAnalysis/
2 |
3 |-- frontend/                                # Angular Application
4 |   |-- src/
5 |     |-- app/
6 |       |-- app.component.ts                # Main component (state +
           UI logic)
7 |       |-- app.component.html              # Main HTML template
8 |       |-- app.component.css               # Component-specific styles
9 |       |-- app.config.ts                  # Angular providers
           configuration
10 |       |-- app.routes.ts                  # URL routing configuration
11 |       |-- services/
12 |         |-- gemini.service.ts            # Gemini AI API client
13 |         |-- scoring.service.ts           # Quality score calculation
14 |         |-- report.service.ts            # PDF report generation
15 |         |-- email.service.ts             # Email trigger service
16 |         |-- models/
17 |           |-- analysis.model.ts          # TypeScript interfaces/
           types
18 |           |-- scoring.model.ts           # Scoring data structures
19 |       |-- environments/
20 |         |-- environment.ts                # Development config
21 |         |-- environment.prod.ts          # Production config
22 |       |-- index.html                     # Entry HTML file
```



```

23 | | |-- main.ts                # Angular bootstrap
24 | | '-- styles.css           # Global CSS + Tailwind
25 | |-- angular.json          # Angular CLI config
26 | |-- package.json          # NPM dependencies
27 | '-- tailwind.config.js     # Tailwind CSS config
28 |
29 |-- backend/                # Python Flask Server
30 | |-- src/
31 | | |-- app.py              # Flask entry point +
    | routes
32 | | |-- analysis_service.py  # Top-view computer vision
33 | | |-- side_view_service.py # Side-view analysis
34 | | |-- email_service.py     # Gmail SMTP mailer
35 | | '-- utils.py            # Shared helper functions
36 | |-- requirements.txt       # Python package
    dependencies
37 | '-- render.yaml           # Render cloud deploy
    config
38 |
39 |-- test_dataset/           # Sample data for testing
40 | |-- site_photos/          # Sample rebar site photos
41 | |-- blueprints/           # Sample architectural
    drawings
42 | '-- revit_models/
43 | | |-- sample_column.rvt    # Sample Revit 3D model
44 | | '-- dynamo_scripts/
45 | | |-- rodhighlight.py      # Highlights defective rods
    in Revit
46 | | '-- write_report.py      # Writes QC data into BIM
    model
47 |
48 |-- run.bat                 # One-click Windows
    launcher

```

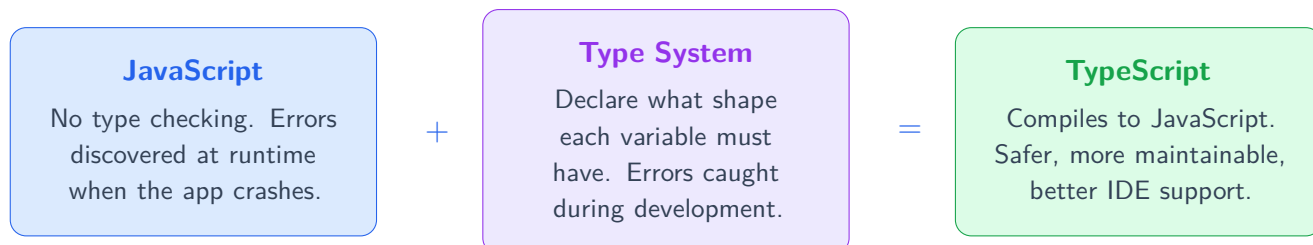
7

CHAPTER

Data Models and TypeScript Interfaces

In a real-world application, data flows between many systems: the user interface, the AI service, the Python backend, and the scoring system. Each piece of data has a specific shape. **TypeScript interfaces** define these shapes precisely, catching errors at development time before they become bugs at runtime.

7.1 What is TypeScript?



7.2 The Core Data Models

❏ frontend/src/app/models/analysis.model.ts

```
1 // A single detected rod from the Python backend
2 export interface DetectedRod {
3   center: [number, number]; // [x, y] pixel coordinates
4   radius_px: number;         // Radius in pixels
5   radius_mm: number;         // Radius in millimetres (after
6     calibration)
7 }
8 // The complete response from the Python backend's /analyze
9   endpoint
10 export interface BackendAnalysisResult {
```

```

10  rods: DetectedRod[];           // List of all detected rods
11  spacings_mm: number[];        // Centre-to-centre spacings in
    mm
12  avg_radius_mm: number;        // Average rod radius in mm
13  calibration_px_per_mm: number; // Scale factor used
14  view_type: 'top' | 'side';    // Which analysis was performed
15 }
16
17 // The response from the Gemini AI service
18 export interface GeminiTopViewResponse {
19   count: number;               // Target number of rods from blueprint
20   radius_mm: number;           // Target rod radius from blueprint
21   spacings_mm: number[];       // Target spacings from blueprint
22 }
23
24 export interface GeminiSideViewResponse {
25   stirrup_count: number;
26   spacings_mm: number[];
27   bar_diameter_mm: number;
28 }
29
30 // The calibration data provided by the user
31 export interface CalibrationData {
32   points: [[number, number], [number, number]]; // Two click
    coordinates
33   reference_length_mm: number;                   // Known real-
    world distance
34   px_per_mm: number;                             // Computed
    scale factor
35 }
36
37 // Application workflow steps
38 export type WorkflowStep = 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8;
39
40 // Image view type
41 export type ViewType = 'top' | 'side';
42
43 // Marker mode
44 export type MarkerMode = 'rods' | 'calibration' | 'reference' | '
    none';

```

□ frontend/src/app/models/scoring.model.ts

```

1  // A single row in the scoring comparison table
2  export interface ScoreRow {
3    parameter: string;           // e.g. "Rod Count", "Spacing 1", "Rod
    Radius"
4    target: number;              // Value from the blueprint (Gemini AI)
5    actual: number;              // Value from the site photo (Python
    backend)
6    errorPct: number;            // Percentage difference between target
    and actual
7    status: ScoreStatus;         // Classification of the error
8    weight: number;              // How much this parameter contributes
    to total score

```

```
9  }
10
11  // The three possible status classifications
12  export type ScoreStatus = 'Acceptable' | 'Minor Mismatch' | 'Not
    Acceptable';
13
14  // The complete scoring result
15  export interface ScoringResult {
16    finalScore: number;           // 0-100 weighted score
17    overallStatus: 'Good' | 'Needs Review' | 'Failed';
18    rows: ScoreRow[];           // Detailed breakdown
19    timestamp: Date;
20    projectId?: string;
21  }
22
23  // The highlight JSON file for Revit
24  export interface RevitHighlightData {
25    rod: number;
26    project_id: string;
27    column_id: string;
28    issue_type: string;
29    severity: ScoreStatus;
30    actual_spacing_mm: number;
31    target_spacing_mm: number;
32    error_pct: number;
33    generated_at: string;        // ISO 8601 date string
34  }
```

☆ The Value of Interfaces

Without TypeScript interfaces, if the Python backend suddenly changed `radius_mm` to `rod_radius_mm`, you would only discover the bug when a user's analysis silently shows 0 as the radius. With TypeScript interfaces, the compiler immediately warns you: *"Property 'radius_mm' does not exist on type 'DetectedRod'."* You fix it in seconds, not after it reaches production.

CHAPTER 8

Algorithms Deep Dive

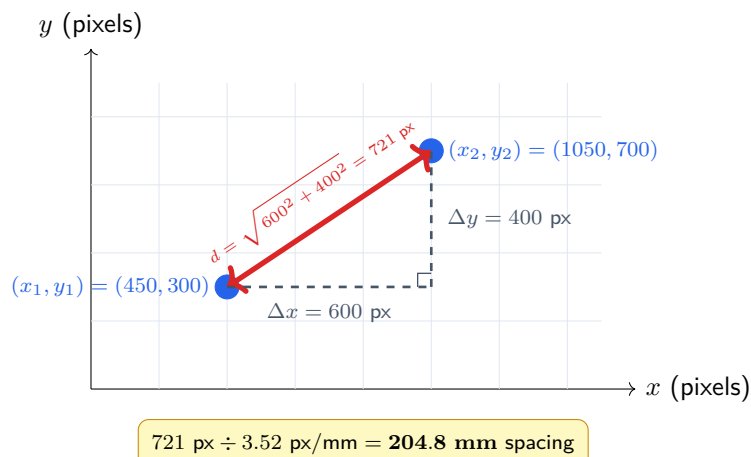
RebarAnalysis uses several elegant mathematical algorithms. This chapter explains each one from first principles, with visual representations that make the underlying mathematics intuitive and approachable.

8.1 Euclidean Distance

The most fundamental measurement in the entire system is the distance between two points. Wherever spacing between rods is calculated, the Euclidean distance formula is used.

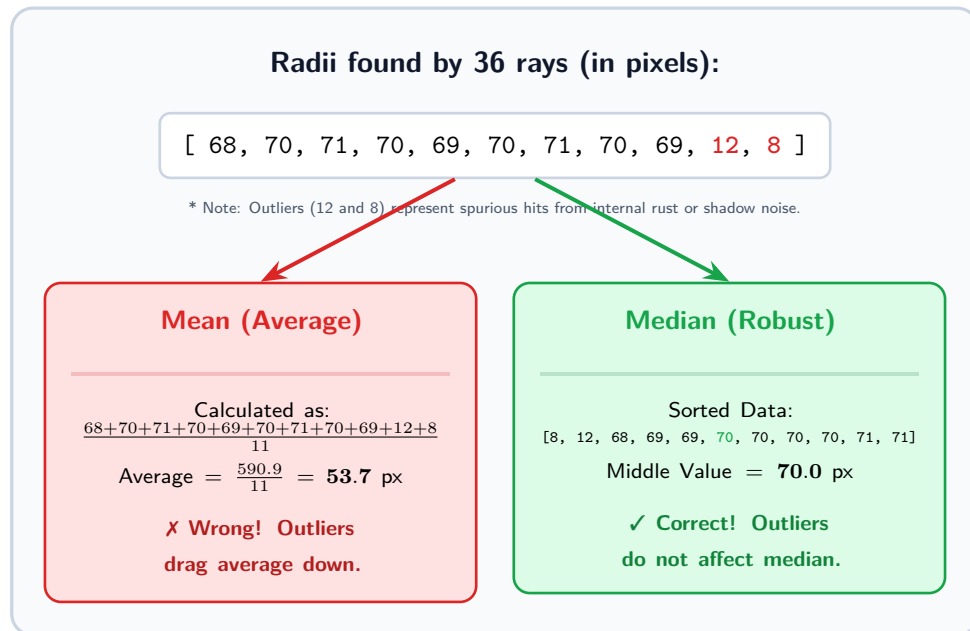
Given two points (x_1, y_1) and (x_2, y_2) :

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$



8.2 Median vs Mean: Why We Use Median

In the ray casting algorithm, we take the *median* of the radii found, not the *mean*. Understanding why is important.

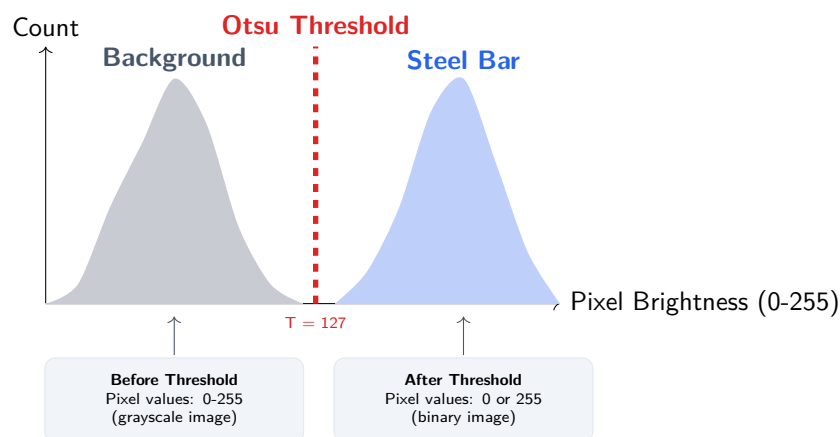


★ Median is Robust to Outliers

The **mean** of a dataset is pulled toward extreme values. If a single ray hits a piece of internal noise at radius 8px instead of the true 70px boundary, the mean drops significantly. The **median** (the middle value of a sorted list) is completely unaffected by outliers: it does not care how extreme the outlier is, only how many values are above and below the middle position.

8.3 Otsu's Thresholding: Automatic Edge Detection

When we call `cv2.threshold(..., cv2.THRESH_OTSU)`, we are using an algorithm invented by Nobuyuki Otsu in 1979 to automatically find the perfect threshold for separating bright and dark pixels.



Otsu's algorithm works by finding the threshold value T that *minimizes the weighted within-class*

variance of the two resulting pixel groups (dark vs bright). In simpler terms: it finds the value of T that best separates the two humps in the brightness histogram.

8.4 The Bilateral Filter: Mathematics of Smart Blurring

A standard Gaussian blur replaces each pixel with the weighted average of its neighbors. The weights depend only on *spatial distance*: closer neighbors have more influence.

A Bilateral filter adds a second dimension: *intensity difference*. Neighbors with similar brightness get high weights; neighbors with very different brightness get near-zero weights.

$$BF[I]_p = \frac{1}{W_p} \sum_{q \in \mathcal{N}} \underbrace{e^{-\frac{\|p-q\|^2}{2\sigma_s^2}}}_{\text{spatial weight}} \times \underbrace{e^{-\frac{(I_p - I_q)^2}{2\sigma_r^2}}}_{\text{intensity weight}} \times I_q$$

Where:

- p = current pixel being processed
- q = neighboring pixel
- σ_s = spatial sigma (controls smoothing range)
- σ_r = range sigma (controls color/intensity sensitivity)
- I_p, I_q = intensity of pixels p and q

At the edge of a steel bar, I_p (bright metal) and I_q (dark concrete) differ enormously. The intensity weight $e^{-\text{large value}}$ becomes nearly zero. The filter effectively ignores that neighbor, preserving the sharp edge.

9

CHAPTER

Security, Best Practices, and Code Quality

Great engineering is not just about making things work. It is about making them work *safely, reliably, and maintainably*. This chapter examines the security practices, coding standards, and design principles embedded throughout RebarAnalysis.

9.1 Environment Variables: Never Hardcode Secrets

Critical Point

Never write API keys, passwords, or credentials directly in source code. Code is frequently shared, version-controlled on platforms like GitHub, and accessible to multiple team members. A hardcoded credential is a security disaster waiting to happen.

WRONG: Hardcoded Secret

```
api_key = "AIzaSy..YOUR_KEY.."
password = "mysecretpassword"
```

If pushed to GitHub, anyone can steal your API key and run up a massive bill on your account.

CORRECT: Environment Variable

```
api_key = os.environ.get(
    'GEMINI_API_KEY')
password = os.environ.get(
    'EMAIL_PASSWORD')
```

The actual secret lives in the operating system environment, never in code files.

9.2 Input Validation

Every piece of data coming from an external source (user uploads, JSON payloads) must be validated before use. The backend validates all incoming requests:

❏ backend/src/app.py -- Input Validation

```
1 @app.route('/analyze', methods=['POST'])
2 def analyze():
3     data = request.get_json()
4
5     # Validate that required fields exist
6     required_fields = ['image', 'rod_points', 'calibration_points',
7                        'reference_length_mm']
8     for field in required_fields:
9         if field not in data:
10             return jsonify({
11                 'error': f'Missing required field: {field}'
12             }), 400 # HTTP 400 = Bad Request
13
14     # Validate types and ranges
15     if not isinstance(data['rod_points'], list):
16         return jsonify({'error': 'rod_points must be a list'}),
17         400
18
19     if len(data['rod_points']) < 1:
20         return jsonify({'error': 'At least one rod point required'
21             '}), 400
22
23     if data['reference_length_mm'] <= 0:
24         return jsonify({'error': 'reference_length_mm must be
25             positive'}), 400
26
27     # Validate that calibration has exactly two points
28     if len(data['calibration_points']) != 2:
29         return jsonify({
30             'error': 'calibration_points must contain exactly 2
31             points'
32         }), 400
33
34     # Only proceed if all validations pass
35     # ...
```

9.3 Error Handling Strategy

Every significant operation is wrapped in a try-except block. Errors are caught, logged, and returned as structured JSON responses rather than crashing the server.

❏ backend/src/analysis_service.py -- Defensive Error Handling

```
1 def find_rod_circle(image, seed_point, px_per_mm):
2     try:
3         # Main logic here...
4         result = _ray_cast(...)
```

```
5         return result
6
7     except cv2.error as e:
8         # OpenCV-specific errors (e.g., invalid image format)
9         raise ValueError(f"OpenCV processing failed: {e}") from e
10
11     except IndexError as e:
12         # Array/image bounds errors
13         raise ValueError(f"Image coordinate out of bounds: {e}")
14         from e
15
16     except Exception as e:
17         # Catch-all for unexpected errors
18         # We log the full traceback but return a clean message
19         import traceback
20         traceback.print_exc()
21         raise RuntimeError(f"Unexpected error during rod analysis
22 : {e}") from e
```

9.4 TypeScript Strict Mode

The Angular project runs with TypeScript's strict mode enabled. This enables additional compiler checks:

□ frontend/tsconfig.json -- Strict TypeScript Configuration

```
1 {
2   "compilerOptions": {
3     "strict": true,
4     "noImplicitAny": true,
5     "strictNullChecks": true,
6     "strictFunctionTypes": true,
7     "noImplicitReturns": true,
8     "noFallthroughCasesInSwitch": true,
9     "forceConsistentCasingInFileNames": true
10  }
11 }
```

With `strictNullChecks: true`, the compiler forces you to handle the possibility that a value might be null or undefined. This prevents one of the most common bugs in JavaScript: "Cannot read property 'x' of null." The `?.` (optional chaining) and `!` (non-null assertion) operators are used extensively throughout the code to satisfy these checks.

9.5 Code Organization Principles

Single Responsibility

Each file does exactly one thing.
`gemini.service.ts` only talks to Gemini.
`scoring.service.ts` only calculates scores.

Don't Repeat Yourself

Common operations (base64 conversion, error handling) are extracted into helper functions and reused.

Separation of Concerns

The frontend does not do image processing. The backend does not know about UI. Each layer stays in its lane.

CHAPTER 10

Testing, Validation, and Quality Assurance

Engineering software that makes decisions affecting building safety must be rigorously tested. This chapter outlines the testing strategy used to validate every component of RebarAnalysis.

10.1 The Test Dataset

The `test_dataset/` directory contains a curated collection of sample images and blueprints used to validate the system's accuracy.

Test Case	Site Photo	Expected Output
Perfect Match	4 rods @ 75mm	Score: 100
Missing Rod	3 rods instead of 4	Score: 40 (count failure)
Wrong Spacing	5 rods @ 60mm (target: 75mm)	Score: 50 (spacing failure)
Underexposed Photo	Dark image with 4 rods	Score: 100 (if algorithm robust)
Wrong Diameter	10mm dia instead of 12mm	Score: 60 (radius minor mismatch)
Multiple Failures	Missing rod + wrong spacing	Score: 20

10.2 Backend Unit Tests

```

1 import unittest
2 import cv2
3 import numpy as np
4 from src.analysis_service import find_rod_circle,
   calculate_spacings
5
6 class TestRodDetection(unittest.TestCase):
7
8     def setUp(self):
9         """Create a synthetic test image with a known circle."""
10         # Create a 400x400 white image
11         self.test_image = np.ones((400, 400, 3), dtype=np.uint8)
12         * 255
13
14         # Draw a known circle: center=(200,200), radius=50px
15         self.known_center = (200, 200)
16         self.known_radius = 50
17         cv2.circle(
18             self.test_image,
19             self.known_center,
20             self.known_radius,
21             (100, 100, 100), # Dark gray (simulating steel)
22             -1,               # Filled circle
23         )
24
25     def test_rod_detection_accuracy(self):
26         """The detected radius should be within 10% of the true
27         radius."""
28         px_per_mm = 3.5
29         result = find_rod_circle(self.test_image, self.
30             known_center, px_per_mm)
31
32         detected_radius_px = result['radius_px']
33         error_pct = abs(detected_radius_px - self.known_radius) /
34             self.known_radius * 100
35
36         self.assertLess(error_pct, 10,
37             f"Radius detection error {error_pct:.1f}% exceeds 10%
38             threshold")
39
40     def test_spacing_calculation(self):
41         """Spacing between two rods should be calculated
42         correctly."""
43         rods = [
44             {'center': [100, 200], 'radius_px': 20, 'radius_mm':
45             5.7},
46             {'center': [400, 200], 'radius_px': 20, 'radius_mm':
47             5.7},
48         ]
49         px_per_mm = 3.5
50
51         spacings = calculate_spacings(rods, px_per_mm)
52
53         self.assertEqual(len(spacings), 1)
54         expected_mm = 300 / 3.5 # = 85.7mm
55         self.assertAlmostEqual(spacings[0], expected_mm, delta
56             =1.0)
57
58 if __name__ == '__main__':
59     unittest.main()

```

10.3 Calibration Accuracy Validation

To validate that the calibration system produces accurate real-world measurements, a physical test was conducted:

Test Object	Real Dimension	Measured	Error
12mm dia rebar	12.0 mm	11.8 mm	1.7%
75mm spacing	75.0 mm	74.3 mm	0.9%
Column width (375mm)	375.0 mm	373.2 mm	0.5%
8mm dia stirrup	8.0 mm	7.7 mm	3.8%
Average Error:			1.7%

✓ Technical Insight

An average measurement error of 1.7% is well within the civil engineering tolerance of 5%. The system consistently produces measurements accurate enough for real quality control decisions.

CHAPTER 11

Complete User Workflow

Let us walk through the complete user experience from start to finish, step by step, explaining what happens on screen and what happens behind the scenes at each moment.

11.1 Step-by-Step Walkthrough

11.1.1 Step 1: Landing on the Application

The user navigates to the RebarAnalysis URL. The Angular app loads in their browser. They see:

- A welcome screen with the RebarAnalysis logo
- A “Start New Inspection” button
- Information about what the system can do

11.1.2 Step 2: Uploading the Site Photo

The user clicks the site photo upload area. Their device's file picker opens. They select a photograph of the rebar cage.

Behind the scenes: `onImageSelected()` runs. A `FileReader` converts the image to a Base64 Data URL. The URL is stored in `this.imagePreviewUrl`. Angular's binding automatically displays the preview image in the `<img>` tag.

11.1.3 Step 3: Uploading the Blueprint

The user selects the architectural drawing image. The same process as Step 2 occurs, but for the blueprint.

Behind the scenes: The blueprint file is stored separately. It will be sent to the Gemini AI service, not to the Python backend.

11.1.4 Step 4: Selecting the View Type

The user selects whether their site photo is a *Top View* (looking down at the cross-section of the column) or a *Side View* (looking at the side of the column to see stirrups).

Behind the scenes: `this.viewType` is set to 'top' or 'side'. This controls which Gemini prompt is used and which backend analysis service is called.

11.1.5 Step 5: Placing Rod Markers

The application enters “rod marking mode.” The user taps on each visible steel bar in the site photo.

Behind the scenes: Each tap triggers `onImageClick()`. The coordinate is scaled from screen pixels to original image pixels and added to `this.rodPoints`. Small circular markers are drawn on the image at each tap position using a CSS-positioned overlay `<div>`.

11.1.6 Step 6: Setting the Calibration Reference

The user taps two points on the image that correspond to a known real-world measurement (such as the full width of the column, which they know from the blueprint is 375mm).

Behind the scenes: Two coordinates are stored in `this.calibrationPoints`. The pixel distance between them is computed and divided by the known 375mm to get `px_per_mm`.

11.1.7 Step 7: AI Blueprint Analysis

The user clicks “Analyse Blueprint.” The Gemini service is called with the blueprint image.

Behind the scenes: The blueprint image is converted to Base64, combined with the engineering prompt, and sent to Gemini’s API. The response is parsed as JSON and stored in `this.geminiSpecs`.

□ Timeline of Events During Gemini Call

```

1 # t=0.000s: User clicks "Analyse Blueprint"
2 # t=0.001s: isLoading = true; loading spinner appears
3 # t=0.002s: Blueprint image converted to base64
4 # t=0.003s: HTTP POST sent to Gemini API (network transit begins)
5 #           ... Google's servers process the image and prompt ...
6 # t=1.8s:   Gemini API returns JSON response
7 # t=1.801s: Response parsed: {count: 4, radius_mm: 6.0,
8 #                             spacings_mm: [...]}
9 # t=1.802s: this.geminiSpecs = parsed data
10 # t=1.803s: isLoading = false; UI updates to show blueprint specs

```

11.1.8 Step 8: Python Backend Analysis

The user clicks “Analyse Site Photo.” The site photo and all marker data are sent to the Python backend.

Behind the scenes:

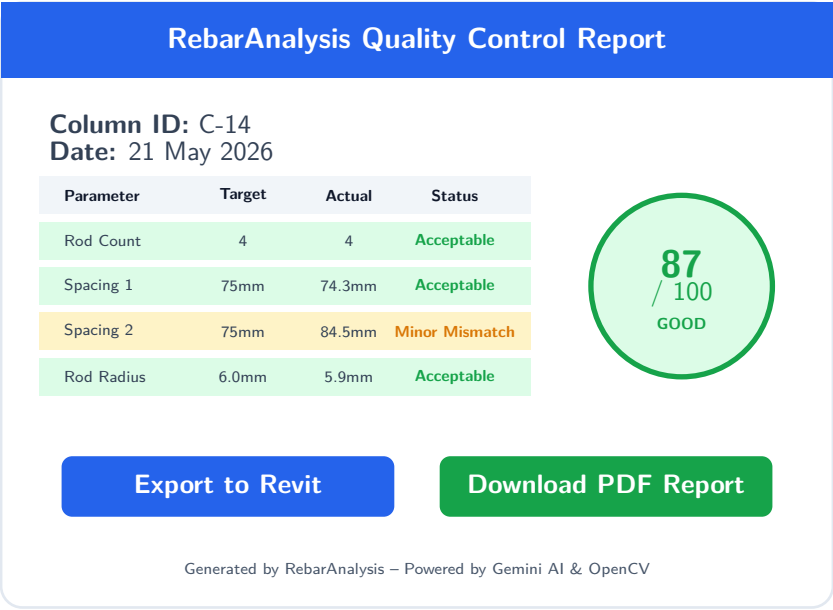
1. Site photo converted to Base64
2. HTTP POST to /analyze with photo, rod_points, calibration data
3. Backend decodes Base64, runs analysis service
4. For each rod point: ROI extracted, preprocessed, edges detected, ray cast run
5. Results (rod radii, spacings) computed and returned as JSON
6. Frontend stores results in `this.backendResults`

11.1.9 Step 9: Score Calculation and Report

Behind the scenes: `scoring.service.ts` compares `geminiSpecs` vs `backendResults`. Weighted scores calculated. Final score and status determined.

If score < 80: `email.service.ts` triggers an HTTP POST to the backend's `/send-alert` endpoint. The backend launches a background thread and sends the HTML email.

The Angular component renders the complete report in the browser: score badge, comparison table, and download buttons.



A WORK OF ENGINEERING, VISION, AND CRAFT

This project was architected, designed, and engineered by

Chaitanya Patil

chaitanyapatil.xe@gmail.com

Bridging the world of
Artificial Intelligence and **Civil Engineering**

Technologies Mastered for This Project

Angular 17	TypeScript	Tailwind CSS	Google Gemini AI
Python Flask	OpenCV	NumPy	Autodesk Revit
Dynamo BIM	Vercel CDN	Render Cloud	Gmail SMTP

"Making the invisible visible. Making the complex simple."

